ZATIQ · আর্কিটেকচার ও বুকিং গাইড · সংস্করণ ৭

হজ বুকিং প্ল্যাটফর্ম

আর্কিটেকচার; এজেন্সি ও সাধারণ ইউজারের বুকিং, দাম আর রুম অ্যালোকেশন; Monorepo বনাম আলাদা রিপো; এবং ক্লায়েন্টের বর্তমান সিস্টেম

সূচিপত্র

পর্ব ১ · সিস্টেম আর্কিটেকচার

1. সংক্ষেপে পরামর্শ
2. আর্কিটেকচার ও ফোল্ডার স্ট্রাকচার
3. কেন Monorepo + Modular Monolith
4. টেক স্ট্যাক (Nuxt না Next.js?)

পর্ব ২ · বুকিং, দাম ও রুম অ্যালোকেশন

5. বুকিং-এর ব্যবসায়িক নিয়ম
6. কে বুক করে, কোন ধরনের বুকিং
7. বুকিং ফ্লো ও স্ট্যাটাস
8. দাম ও WhatsApp কাস্টম প্রাইস
9. রুম অ্যালোকেশন অ্যালগরিদম
10. একসাথে অনেক বুকিং: লক, হোল্ড ও কোটা
11. এক্সট্রা বেড
12. বুকিং বদল, বাতিল ও চেক-ইন
13. পাসপোর্টের ছবি থেকে স্বয়ংক্রিয় তথ্য
14. অকুপেন্সি ক্যালেন্ডার
15. অ্যাডমিন সেটিংস
16. ডাটা মডেল ও ডাটাবেস নিয়ম
17. পুরো উদাহরণ

পর্ব ৩ · রিপো কৌশল

18. Monorepo বনাম আলাদা রিপো: বিস্তারিত তুলনা

পর্ব ৪ · ক্লায়েন্টের বর্তমান সিস্টেম

19. বর্তমান বিজনেস লজিক
20. নতুন নিয়মগুলো বর্তমান সিস্টেমে কীভাবে বসবে
21. বাইরের ইউজার আসার আগে যা লাগবে
22. প্রস্তাবিত বানানোর ক্রম
23. ক্লায়েন্টের সাথে যা নিশ্চিত করতে হবে

প্রস্তুত: Md Sumon, Zatiq · অক্টোবর ২০২৬

১. সংক্ষেপে পরামর্শ

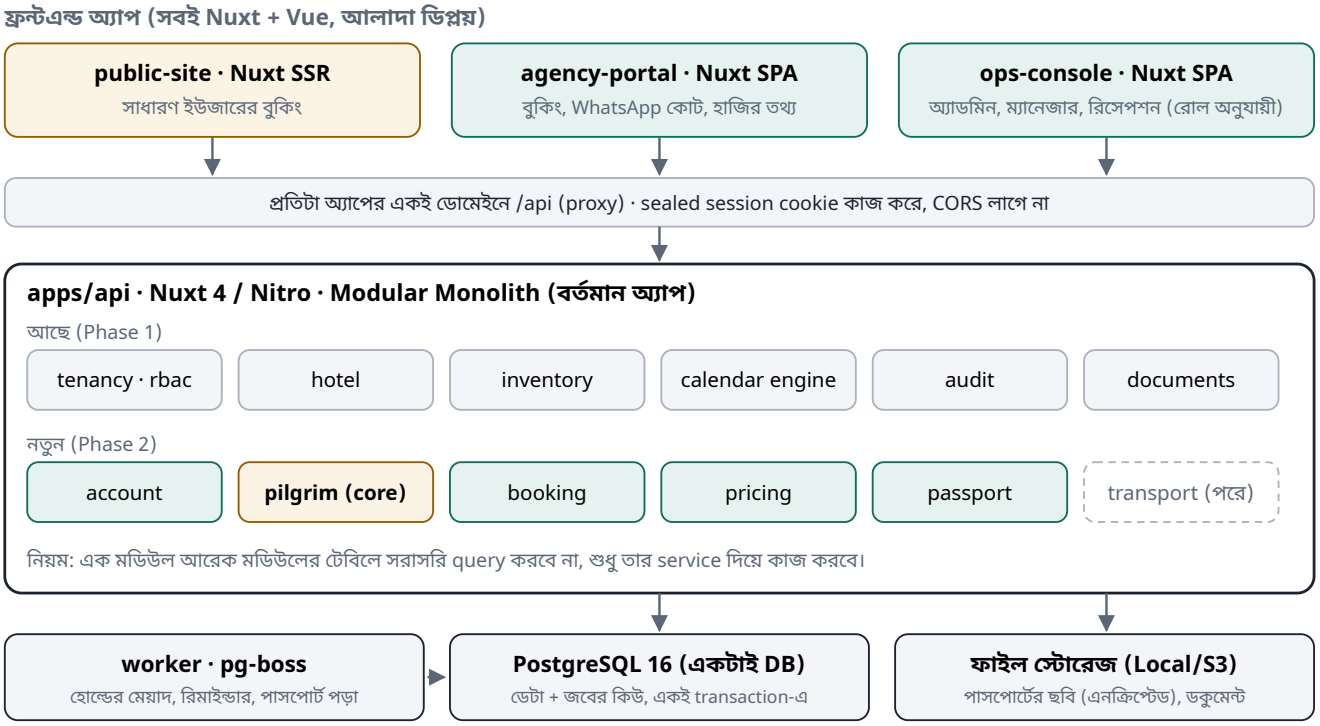
> **ক্লায়েন্টের বর্তমান Nuxt 4 / Nitro সিস্টেমই ভিত্তি থাকবে, নতুন করে লেখা হবে না।** এটা Modular Monolith, PostgreSQL আর Drizzle দিয়ে বানানো, আর এর Phase 1 শেষ। বুকিং, দাম, হাজি আর পাসপোর্ট এর ভেতরেই নতুন মডিউল হিসেবে যোগ হবে।
>
> কোড থাকবে একটা monorepo-তে: একটা ব্যাকএন্ড (বর্তমান অ্যাপ), তিনটা Nuxt ফ্রন্টএন্ড, আর শেয়ার করা প্যাকেজ। Microservices বা আলাদা রিপোর দরকার নেই।

> **সংস্করণ ৫ থেকে:** আগের সংস্করণগুলোর এই পর্বে NestJS, React আর BullMQ-এর কথা ছিল, যেগুলো ক্লায়েন্টের কোড দেখার আগে লেখা। ফলে পর্ব ১ আর পর্ব ৪ পরস্পরবিরোধী ছিল। এই সংস্করণে পর্ব ১ বর্তমান স্ট্যাক অনুযায়ী নতুন করে লেখা।

সিস্টেম কী করবে (সারসংক্ষেপ)

- মক্কা ও মদিনার ৫–৬টা হোটেল। বুক করতে পারবে তিন পক্ষ: **এজেন্সি** (নিজের পোর্টাল থেকে), **সাধারণ ইউজার** (পাবলিক ওয়েবসাইট থেকে), আর **হোটেল ম্যানেজার** (এজেন্সির পক্ষে)।
- বুকিং দুই ধরনের: **ফ্যামিলি** (স্বামী-স্ত্রী বা পরিবার এক রুমে, পুরুষ-মহিলা আলাদা করা হয় না) আর **গ্রুপ** (পুরুষ আর মহিলা আলাদা রুমে)।
- সাধারণ ইউজার পাবলিক দামে বুক করবে। এজেন্সি WhatsApp-এ কথা বলে কম দামের কাস্টম প্রাইস পাবে।
- এজেন্সি হাজির তথ্য ছাড়াই, শুধু নিজের তথ্য দিয়ে আগে থেকে রুম বুক করতে পারবে। হাজির তথ্য পরে এজেন্সি বা হোটেল ম্যানেজার দেবে। পাসপোর্টের ছবি ড্র্যাগ-অ্যান্ড-ড্রপ করলেই তথ্য বসে যাবে।
- গ্রুপে এক রুমে এক এজেন্সি আর এক লিঙ্গ। একই এজেন্সি একই তারিখে আবার বুক করলে আগে নিজের রুমের খালি সিট ভরবে।
- দাম, রুম-মিক্স, কোটা আর এক্সট্রা বেডের নিয়ম অ্যাডমিন প্যানেল থেকে।
- ম্যানেজার আর অ্যাডমিন একটা বছর/মাস/দিনের ক্যালেন্ডারে দেখবে কোন দিনে কোন ফ্লোরের কোন রুম বুকড বা খালি, কারা আছে।
- ট্রান্সপোর্টসহ ভবিষ্যতের সব সিস্টেম একই ব্যাকএন্ড আর ডাটাবেস ব্যবহার করবে।

২. আর্কিটেকচার ও ফোল্ডার স্ট্রাকচার




```
hajj-umrah-hotel-ops/      ← একটা monorepo (pnpm workspaces + Turborepo)
├── apps/
│   ├── api/               ← বর্তমান Nuxt/Nitro অ্যাপ (API + worker)
│   ├── public-site/       ← সাধারণ ইউজারের বুকিং (Nuxt SSR, SEO)
│   ├── agency-portal/     ← এজেন্সি: বুকিং, WhatsApp কোট, হাজির তথ্য (Nuxt SPA)
│   └── ops-console/       ← অ্যাডমিন + হোটেল ম্যানেজার + রিসেপশন, রোল অনুযায়ী (Nuxt SPA)
└── packages/
    ├── db/                ← Drizzle schema + migrations (বর্তমান db/ এখানে আসবে)
    ├── contracts/         ← Zod স্কিমা ও টাইপ (বর্তমান shared/schemas এখানে আসবে)
    └── ui/                ← Nuxt UI v4-এর ওপর শেয়ার করা Vue কম্পোনেন্ট
```

> **কেন তিনটা ফ্রন্টএন্ড, পাঁচটা নয়:** অ্যাডমিন, হোটেল ম্যানেজার আর রিসেপশন একই RBAC ব্যবহার করে, তাই এক অ্যাপে রোল অনুযায়ী মেনু। আলাদা অ্যাপ বানালে লেআউট আর লগইন দুইবার লিখতে হতো। ট্রান্সপোর্ট পরে ops-console-এর একটা অংশ হবে।
>
> যে ভাগটা দরকারি: **বাইরের ইউজার (এজেন্সি) আর ভেতরের স্টাফ আলাদা অ্যাপে**, যাতে লগইন, কুকি আর অ্যাক্সেস আলাদা থাকে। পাবলিক সাইট আলাদা, কারণ সেখানে SSR আর SEO লাগে।

- **একই ডোমেইনে API:** প্রতিটা অ্যাপ `/api` নিজের ডোমেইনেই proxy করবে। তাতে বর্তমান sealed session cookie কাজ করে, CORS লাগে না।
- **worker:** pg-boss-এর হালকা কাজ (হোল্ডের sweep, রিমাইন্ডার) api-এর পাশেই চলতে পারে। কিন্তু **পাসপোর্টের OCR প্রথম দিন থেকেই আলাদা worker প্রসেসে**, কারণ এটা CPU-ভারী। একসাথে ৩০টা ছবি এলে নইলে সব API রিকোয়েস্ট ধীর হয়ে যাবে।

- **আরও সহজ শুরু চাইলে:** প্রথমে একটা Nuxt অ্যাপেই `routeRules` দিয়ে (পাবলিক পেজ SSR, পোর্টাল SPA) চালানো যায়। তবে তখন এজেন্সি আর স্টাফের লগইন আলাদা রাখার নিরাপত্তা-সুবিধা থাকে না। এই ডকে monorepo ধরে নেওয়া হয়েছে।

ব্যাকএন্ডের মডিউল

মডিউল	অবস্থা	দায়িত্ব
`tenancy · rbac`	আছে	organization, ইউজার, রোল, অনুমতি। নতুন: এজেন্সি আর সাধারণ ইউজারের জন্য `AgencyScope` ও `CustomerScope`।
`hotel`	আছে	হোটেল, ফ্লোর, হোটেলের সেটিং
`inventory`	আছে	রুম টাইপ, রুম, ভার্সনসহ ক্যাপাসিটি, সিজনাল ওভাররাইড, ব্লক, ক্যালেন্ডার ইঞ্জিন। নতুন: যেকোনো পরিবর্তনের আগে বুকিং-এর সাথে সংঘর্ষ চেক।
`audit · documents`	আছে	অডিট লগ, ফাইল স্টোরেজ। নতুন: পাসপোর্টের তথ্য কে দেখেছে তার লগ।
`account`	নতুন	এজেন্সি (অনুমোদনসহ) আর সাধারণ ইউজার
`pilgrim`	নতুন	হাজির তথ্য। এটা **core মডিউল**, সব সিস্টেম এটাকে রেফার করবে।
`booking`	নতুন	ফ্যামিলি ও গ্রুপ বুকিং, অ্যালোকেশন, হোল্ড, বদল, বাতিল, প্রতি হাজির চেক-ইন
`pricing`	নতুন	পাবলিক দাম, এজেন্সির দাম, WhatsApp কোট, প্রতি রাতের দাম
`passport`	নতুন	পাসপোর্টের ছবি থেকে তথ্য পড়া (worker-এ চলে)
`transport`	পরে	ট্রিপ, গাড়ি, কোন হাজি কোথায় যাচ্ছে

ভবিষ্যতে নতুন কোনো সিস্টেম আনলে (যেমন ভিসা বা ফ্লাইট) শুধু একটা নতুন মডিউল আর ops-console-এ একটা নতুন অংশ যোগ করলেই হবে। `pilgrim` টেবিল একটাই থাকবে, তাই একজন হাজির সব তথ্য এক জায়গায় পাবেন।

৩. কেন Monorepo + Modular Monolith

বিষয়	Monorepo + Modular Monolith	আলাদা রিপো / Microservices
টাইপ শেয়ার (ফ্রন্ট ↔ ব্যাক)	সরাসরি, `packages/contracts` থেকে	npm package পাবলিশ করতে হয়, ভার্সন মিসম্যাচ হয়
বুকিং + হাজি একসাথে সেভ	একটা DB transaction-এ হয়ে যায়	distributed transaction লাগে, অনেক জটিল
জব আর বুকিং একসাথে সেভ	pg-boss দিয়ে একই transaction-এ	আলাদা কিউ সার্ভিসে outbox প্যাটার্ন লাগে
ছোট টিমের জন্য	খুব উপযুক্ত	DevOps-এর খরচ অনেক বেশি
পারফরম্যান্স	ইন-প্রসেস কল, নেটওয়ার্ক hop নেই	প্রতিটা সার্ভিস কলে latency যোগ হয়
পরে আলাদা করা	মডিউলের সীমানা পরিষ্কার থাকলে সহজে আলাদা করা যায়	আগে থেকেই আলাদা

> **একটা নিয়ম মানবেন:** এক মডিউল আরেক মডিউলের টেবিলে সরাসরি query করবে না, শুধু ওই মডিউলের service দিয়ে কাজ করবে। তাহলে পরে কখনো, যেমন transport বড় হয়ে গেলে, ওটাকে আলাদা সার্ভিস বানাতে পারবেন।
>
> Monorepo হলেও ডিপ্লয় আলাদা আলাদা হবে। প্রতিটা অ্যাপ নিজের মতো ডিপ্লয় হবে, Turborepo শুধু যেটা বদলেছে সেটাই build করবে।

৪. টেক স্ট্যাক

স্তর	পছন্দ	কারণ
ফ্রেমওয়ার্ক	Nuxt 4 / Nitro, TypeScript	বর্তমান সিস্টেম। ব্যাকএন্ড আর তিনটা ফ্রন্টএন্ড একই ফ্রেমওয়ার্কে।
UI	Vue + Nuxt UI v4	অনুমোদিত UI ডিরেকশন; `packages/ui`-তে শেয়ার করা
পাবলিক সাইট	Nuxt SSR	সার্চে আসা আর দ্রুত প্রথম লোড
ড্যাশবোর্ড	Nuxt SPA (`ssr: false`)	লগইন করা, SEO লাগে না
ডাটাবেস	PostgreSQL 16 + `btree_gist`	transaction, row lock, `EXCLUDE` constraint
ORM	Drizzle	আগে থেকেই ব্যবহার হচ্ছে
ভ্যালিডেশন	Zod	সার্ভার আর ক্লায়েন্টে একই স্কিমা
অথেনটিকেশন	nuxt-auth-utils (sealed cookie) + Argon2id	বর্তমান ব্যবস্থা। সাথে যোগ হবে OTP আর স্টাফের 2FA (সেকশন ২১)।
ব্যাকগ্রাউন্ড কাজ	pg-boss	Postgres-এই চলে। "রিমাইন্ডার পাঠাও" ধরনের জব বুকিং-এর সাথে একই transaction-এ সেভ হয়। (একই transaction-এ জব সেভ করা আগে পরীক্ষা করে নিশ্চিত করতে হবে, সেকশন ১০)
ফাইল	`document_asset` (Local/S3)	বর্তমান ব্যবস্থা; পাসপোর্টের জন্য এনক্রিপশন যোগ
Cache	আপাতত নেই	cache বা rate limit-এ সত্যিই দরকার হলে তখন Redis। বুকিং বা হোল্ড কখনো Redis-এ নয়, সবসময় Postgres-এ।
টেস্ট	Vitest	ইউনিট, ইন্টিগ্রেশন, আসল সার্ভারে HTTP
হোস্টিং	সৌদির ভেতরের ক্লাউড রিজিয়ন বিবেচনা + CDN	পাসপোর্টের তথ্য সৌদির হোটেলের জন্য প্রসেস হয়। বাহরাইন বা UAE-তে রাখাও PDPL-এ দেশের বাইরে ডেটা পাঠানো, তাই আইনি যাচাই লাগবে (সেকশন ২১)।

Nuxt না Next.js?

বিষয়	Nuxt রাখা	Next.js-এ যাওয়া
বর্তমান কোড	Phase 1-এর ব্যাকএন্ড, টেস্ট আর অথ সব থাকে	হয় ব্যাকএন্ড নতুন করে লিখতে হবে, নয়তো দুইটা ফ্রেমওয়ার্ক একসাথে চালাতে হবে
শেয়ার করা UI	একটাই Vue কম্পোনেন্ট লাইব্রেরি	Vue আর React-এর কম্পোনেন্ট শেয়ার করা যায় না
টিম	বর্তমান টিম এটাতেই কাজ করছে	নতুন করে শিখতে হবে
SEO ও পারফরম্যান্স	Nuxt SSR যথেষ্ট	পার্থক্য সামান্য, কোনো বাধা নয়
ডেভেলপার পাওয়া	সাধারণত Vue ডেভেলপার তুলনামূলক কম	React ডেভেলপার বেশি পাওয়া যায়

সিদ্ধান্ত: Nuxt। Next.js-এর একমাত্র সত্যিকারের সুবিধা হলো ডেভেলপার বেশি পাওয়া যায়। কিন্তু তার জন্য শেষ হওয়া Phase 1 ফেলে দেওয়া বা দুইটা ফ্রেমওয়ার্ক চালানোর খরচ অনেক বেশি। ওয়েবসাইটের গতি ফ্রেমওয়ার্কের ওপর নয়, ডাটাবেস ডিজাইন, query আর ফ্রন্টএন্ডের bundle size-এর ওপর নির্ভর করে।

পাসপোর্টের তথ্য রাখবেন বলে সৌদির ডেটা প্রটেকশন আইন (PDPL) মেনে চলতে হবে (সেকশন ১৩)।

৫. বুকিং-এর ব্যবসায়িক নিয়ম

ক্লায়েন্টের সাথে আলোচনা থেকে চূড়ান্ত নিয়মগুলো নিচে। পরের সব অংশ (ফ্লো, দাম, অ্যালগরিদম, পাসপোর্ট, ডাটা মডেল) এই নিয়মগুলোর ওপর দাঁড়ানো।

#	নিয়ম	বিস্তারিত
কে বুক করে, কোন দামে		
১	তিন পক্ষ বুক করতে পারে	**এজেন্সি** (নিজের পোর্টাল থেকে), **সাধারণ ইউজার** (পাবলিক ওয়েবসাইট থেকে), আর **হোটেল ম্যানেজার/সেলার** (এজেন্সির পক্ষে)।
২	সাধারণ ইউজার পাবলিক দামে	ওয়েবসাইটে যে দাম দেখানো, সেই দামে বুকিং অনুরোধ পাঠাবে। অনলাইন পেমেন্ট না আসা পর্যন্ত ম্যানেজার নিশ্চিত করলে তবেই কনফার্ম।
৩	এজেন্সি সবসময় কম দামে	এজেন্সি অনেক হাজি আনে, তাই তার দাম পাবলিক দামের চেয়ে কম। যেমন সাধারণ ইউজারের ৩৮০ হলে এজেন্সির ৩৫০।
৪	WhatsApp-এ কাস্টম প্রাইস	এজেন্সি পোর্টালে "WhatsApp-এ দাম জিজ্ঞেস করুন" বাটন। সেলার/ম্যানেজার কথা বলে একটা কাস্টম দাম দেবে, সেই দামে এজেন্সি বুক করবে। **কোট রুম আটকায় না:** মেয়াদের মধ্যে রুম খালি থাকলে তবেই সেই দামে বুক হবে।
বুকিং-এর ধরন		
৫	শুরুতেই প্রশ্ন: ফ্যামিলি না গ্রুপ?	বুকিং শুরুর আগে বুকিং-এর ধরন বাছতে হবে।
৬	ফ্যামিলি	পুরুষ-মহিলা আলাদা করা হবে না। স্বামী-স্ত্রী বা পরিবার এক রুমে থাকতে পারবে। শুধু মোট কতজন, সেটা দেবে। রুম হিসেবে বুক হবে, আর রুমগুলো শুধু ওই বুকিং-এর।
৭	গ্রুপ	কয়জন পুরুষ, কয়জন মহিলা, কয়জন ১০ বছরের কম বয়সী শিশু (কার সাথে থাকবে, ডিফল্ট মায়ের সাথে) আর কয়জন কোলের বাচ্চা, আলাদা দিতে হবে। ১০ বছর বা তার বেশি বয়সী পুরুষ আর মহিলা কখনো এক রুমে থাকবে না।
রুম বরাদ্দ		
৮	এক রুমে এক মালিক	গ্রুপে একটা রুমে এজেন্সি A-র একজন হাজিও থাকলে ওই তারিখে রুমটা অন্য কেউ পাবে না। ফ্যামিলির রুম ওই বুকিং ছাড়া আর কেউ পাবে না।
৯	নিজের রুম আগে, শুধু একই তারিখে	একই এজেন্সির নতুন গ্রুপ বুকিং-এর চেক-ইন আর চেক-আউট আগের বুকিং-এর সাথে **হুবহু এক** হলে, আগে আগের রুমের খালি সিট ভরবে। তারিখ আলাদা হলে, বা অন্য এজেন্সি হলে, নতুন রুম।
১০	রুম-মিক্স অ্যাডমিন ঠিক করবে	গ্রুপের আকার অনুযায়ী কোন সাইজের রুম সর্বোচ্চ কয়টা। যেমন ২০ জনের গ্রুপে ৫ বা তার বেশি সিটের রুম সর্বোচ্চ ২টা (হজে ৬ সিট হয়ে যাওয়া রুমও এতে গোনা), বাকিগুলো ৪, ৩, ২, ১ জনের।
১১	এজেন্সির কোটা	প্রতি হোটেলে, প্রতি সিজনে একটা এজেন্সি এক রাতে সর্বোচ্চ কতজন রাখতে পারবে, নির্দিষ্ট সংখ্যায়। যেমন হজ সিজনে ৩০ জন।
১২	এক্সট্রা বেড	সিট কম পড়লে, অথবা মাত্র ১–২ জনের জন্য পুরো একটা রুম নিতে হলে, নিজের রুমে এক্সট্রা বেড। সীমা ও দাম অ্যাডমিন ঠিক করবে।
১৩	ক্যাপাসিটি তারিখভেদে বদলায়	হজ বা রমজানে রুমের ক্যাপাসিটি বাড়ে। খালি সিট হিসাব হবে পুরো থাকার সময়ের প্রতিটা রাত মিলিয়ে।
হাজির তথ্য		

১৪	এজেন্সি তথ্য ছাড়াই বুক করতে পারবে	শুধু এজেন্সির পুরো তথ্য, কয়টা রুম আর কয়জন হাজি দিয়ে আগে থেকে বুক করা যাবে। হাজির পাসপোর্ট লাগবে না।
১৫	সাধারণ ইউজারের অন্তত একজন	নিজে বুক করলে অন্তত একজনের পাসপোর্টসহ পুরো তথ্য দিতে হবে।
১৬	তথ্য দুই দিক থেকেই দেওয়া যাবে	বাকি হাজির তথ্য এজেন্সি নিজে দিতে পারবে, অথবা হাজিরা হোটেলে পৌঁছালে ম্যানেজার/রিসেপশন দেবে। এক এজেন্সি অন্য এজেন্সির দেওয়া হাজির তথ্য কখনো দেখবে না।
১৭	পাসপোর্টের ছবি দিলেই তথ্য বসে যাবে	ছবি ড্র্যাগ-অ্যান্ড-ড্রপ করলে সিস্টেম ছবি পড়ে সব তথ্য বসিয়ে দেবে। সেভের আগে একটা রিভিউ ফর্মে যাচাই।
১৮	প্রতিটা হাজির চেক-ইন আলাদা	বুকিং নম্বর আর QR কোডসহ রিসিপ্ট। হাজিরা আলাদা সময়ে আসতে পারে, তাই প্রতিজন আলাদাভাবে চেক-ইন হবে। চেক-ইনের আগে তার পুরো তথ্য বাধ্যতামূলক। ২ বছরের কম বয়সী শিশুরও slot থাকবে (তথ্য আর চেক-ইন লাগবে), কিন্তু সিট, কোটা বা দামে গোনা হবে না।
১৯	বুকিং বদল ও বাতিল	হাজি যোগ বা বাদ, তারিখ বদল, রুম বদল, বাতিল, no-show, আগে চেক-আউট: নিয়ম সেকশন ১২-এ।
ব্যবস্থাপনা		
২০	অকুপেন্সি ক্যালেন্ডার	বছর, মাস আর দিনের ভিউ। যেকোনো দিনে কোন ফ্লোরের কোন রুম বুকড বা খালি, কয়জন হাজি, কোন এজেন্সির নাকি ফ্যামিলি। শুধু ম্যানেজার আর অ্যাডমিন দেখবে।

সংস্করণ ৭-এ যা বদলেছে (তৃতীয় কোড রিভিউয়ের পরে):

- হাজির তথ্য: স্টাফের একটা master রেকর্ড, আর প্রতিটা বুকারের নিজের কপি। এক এজেন্সি আরেক এজেন্সির তথ্য দেখতে বা বদলাতে পারে না (সেকশন ১৬)।
- প্রতিটা বুকিং-এর নিজের `booking_room` row, composite FK দিয়ে আটকানো। সিট গোনার নিয়ম এক জায়গায়: `room_night_occupancy` view।
- অ্যালগরিদমের দুইটা ভুল ঠিক: এক্সট্রা বেডের অপশন হারানো, আর হজে ৬ সিটের রুম সীমার বাইরে চলে যাওয়া। এবার আসল রুমের সব সম্ভাব্য সেটের সাথে মিলিয়ে টেস্ট করা (সেকশন ৯.৪)।
- দেখানো দামে খালি সিটের দাম নেই; রুম বাছার পরে দাম আবার হিসাব।
- গ্রুপ বুকিং-এ ১০ বছরের কম বয়সী শিশু আর কোলের বাচ্চার আলাদা সংখ্যা; বয়স হিসাব হয় জন্মতারিখ থেকে।

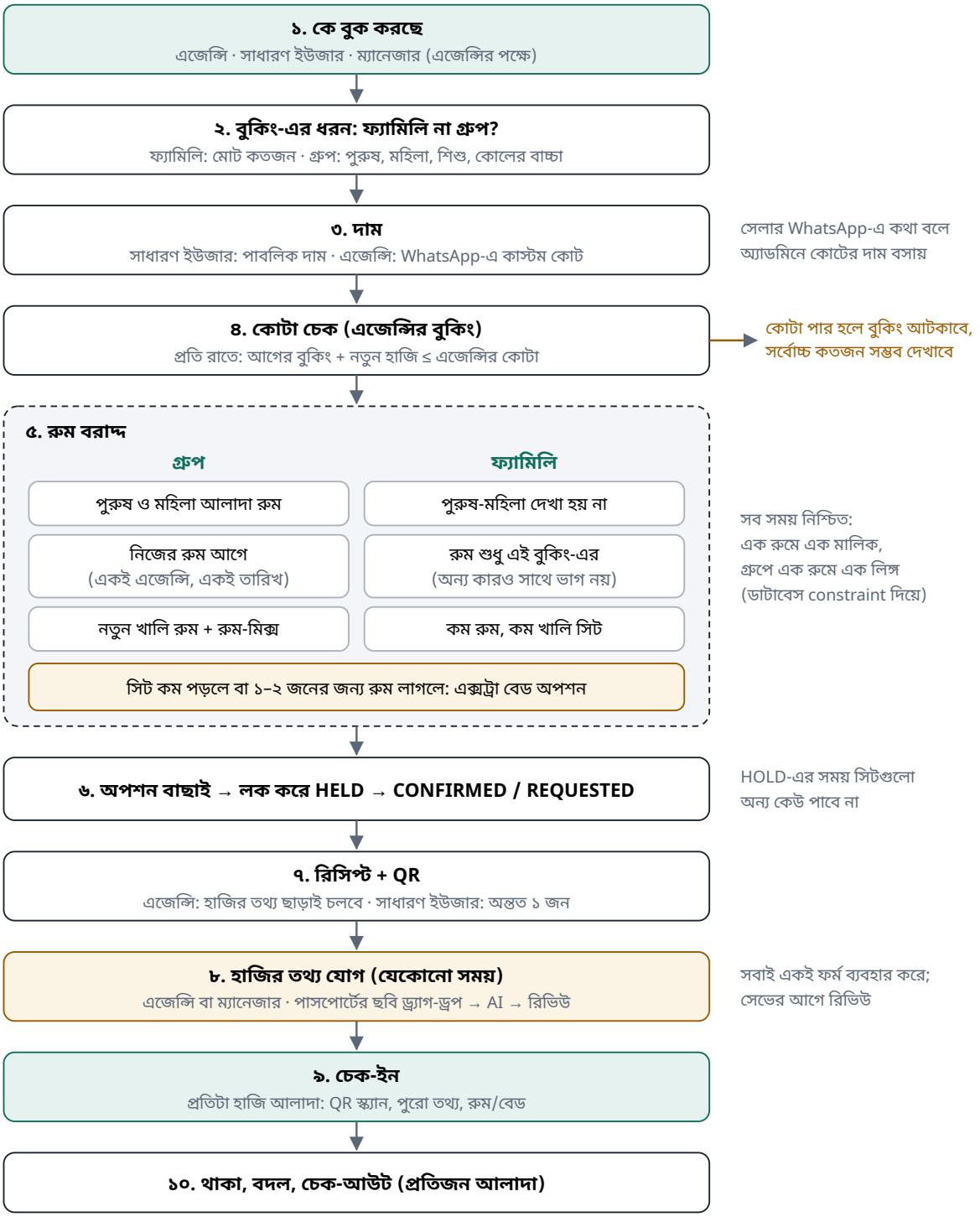
৬. কে বুক করে, কোন ধরনের বুকিং

বিষয়	এজেন্সি	সাধারণ ইউজার	ম্যানেজার/সেলার
কোথা থেকে	এজেন্সি পোর্টাল (লগইন, অ্যাডমিনের অনুমোদিত অ্যাকাউন্ট)	পাবলিক ওয়েবসাইট	অ্যাডমিন/হোটেল প্যানেল, এজেন্সির পক্ষে
দাম	কাস্টম কোট (WhatsApp), না থাকলে এজেন্সির ডিফল্ট দাম	পাবলিক দাম	যে এজেন্সির পক্ষে, তার কোট বা দাম
বুকিং-এর সময় হাজির তথ্য	লাগবে না	অন্তত ১ জনের পাসপোর্টসহ	লাগবে না
কনফার্ম কীভাবে	সরাসরি: হোল্ড → কনফার্ম	অনুরোধ (REQUESTED) → ম্যানেজার নিশ্চিত করলে কনফার্ম। একসাথে ১টা খোলা অনুরোধ।	সরাসরি
কোটা	প্রযোজ্য	না (তবে প্রতি বুকিং-এ সর্বোচ্চ রুমের সীমা)	এজেন্সির কোটা প্রযোজ্য
পরে তথ্য কে দেবে	এজেন্সি বা হোটেল ম্যানেজার/ রিসেপশন	ইউজার নিজে বা রিসেপশন	ম্যানেজার বা এজেন্সি

ফ্যামিলি বনাম গ্রুপ

বিষয়	ফ্যামিলি	গ্রুপ
যা দিতে হয়	মোট কতজন (চাইলে কোন সাইজের কয়টা রুম)	কয়জন পুরুষ, মহিলা, ১০ বছরের কম শিশু (কার সাথে), আর কোলের বাচ্চা
পুরুষ-মহিলা	আলাদা করা হয় না	সবসময় আলাদা রুম
রুম ভাগাভাগি	রুম শুধু এই বুকিং-এর, অন্য কারও সাথে নয়	একই এজেন্সির একই তারিখের বুকিং খালি সিট ভাগ করতে পারে
রুম-মিক্স নিয়ম	প্রযোজ্য নয়, শুধু কম রুম আর কম খালি সিট	প্রযোজ্য
সাধারণত কে	সাধারণ ইউজার (এজেন্সিও পারবে)	এজেন্সি (সাধারণ ইউজারও পারবে)

৭. বুকিং ফ্লো: শুরু থেকে চেক-আউট



বুকিং-এর স্ট্যাটাস

স্ট্যাটাস	মানে
`HELD`	অপশন বাছার সাথে সাথে রুম লক করে সিট আটকানো। ১০ মিনিটে কনফার্ম না হলে `EXPIRED`।
`REQUESTED`	সাধারণ ইউজারের অনুরোধ (পেমেন্ট ছাড়া)। সিট আটকে থাকে নির্দিষ্ট সময় (যেমন ২৪ ঘণ্টা)। ম্যানেজার নিশ্চিত করলে `CONFIRMED`, নইলে `EXPIRED`।
`CONFIRMED`	বুকিং পাকা। রিসিপ্ট আর QR তৈরি। এজেন্সির বুকিং-এ হাজির তথ্য তখনো পুরো খালি থাকতে পারে।
`IN_HOUSE`	অন্তত একজন হাজি চেক-ইন করেছে।
`COMPLETED`	সবাই চেক-আউট করেছে বা no-show।
`CANCELLED` / `EXPIRED`	বাতিল বা মেয়াদ শেষ। সিটগুলো আবার খালি।

প্রতিটা হাজির (slot) আলাদা স্ট্যাটাস: `PENDING_DETAILS` (তথ্য বাকি) → `READY` (তথ্য পূর্ণ) → `CHECKED_IN` → `CHECKED_OUT`, অথবা `NO_SHOW` / `REMOVED`।

বুকিং-এর পাশে আলাদা একটা হিসাব দেখাবে: **"হাজির তথ্য: ১২/৩০ সম্পূর্ণ"**। এজেন্সি আর ম্যানেজার দুজনেই এক নজরে দেখবে কতজনের তথ্য বাকি।

হাজির তথ্য কীভাবে থাকবে

- বুকিং-এ যতজন হাজি, ততগুলো **guest slot** তৈরি হবে। গ্রুপে প্রতিটা slot-এর লিঙ্গ আর রুম আগে থেকেই ঠিক থাকবে। ফ্যামিলিতে শুধু রুম ঠিক থাকবে।
- এজেন্সির বুকিং-এ সব slot শুরুতে `PENDING_DETAILS` থাকতে পারে। সাধারণ ইউজারের বুকিং-এ অন্তত একটা slot পূর্ণ থাকবে।
- এজেন্সি পোর্টাল আর হোটেল প্যানেল, দুই জায়গাতেই একই "হাজির তথ্য যোগ" স্ক্রিন। যে-ই তথ্য দিক, অডিট লগে নাম থাকবে।
- গ্রুপে পাসপোর্টের লিঙ্গ আর slot-এর লিঙ্গ না মিললে সিস্টেম আটকে দেবে, আর slot বদলের পরামর্শ দেবে। ব্যতিক্রম: ১০ বছরের কম বয়সী শিশু।
- **বয়স হাতে বাছা হবে না।** জন্মতারিখ থেকে চেক-ইনের দিনের বয়স হিসাব করে `age_group` ঠিক হবে। বুকিং-এর সময় "১০ বছরের কম" বলা কোনো ছেলের পাসপোর্টে যদি পরে দেখা যায় সে ১১, তাহলে সেটা লিঙ্গ না মেলার মতোই ধরা হবে: আটকে দেবে, আর রুম বদলের পরামর্শ।
- একজন হাজি (`pilgrim`) একবারই তৈরি হবে। পরের বুকিং বা ট্রান্সপোর্টে একই রেকর্ড ব্যবহার হবে।

৮. দাম ও WhatsApp কাস্টম প্রাইস

তিন স্তরের দাম

দাম	কার জন্য	কে ঠিক করে
পাবলিক দাম	সাধারণ ইউজার। হোটেল, রুম টাইপ আর সিজন অনুযায়ী।	অ্যাডমিন
এজেন্সির ডিফল্ট দাম (ঐচ্ছিক)	অনুমোদিত সব এজেন্সি, কোট না থাকলে	অ্যাডমিন
কাস্টম কোট	নির্দিষ্ট এজেন্সি, হোটেল, তারিখ আর হাজির সংখ্যার জন্য	সেলার/ম্যানেজার, WhatsApp-এ আলোচনার পরে

কোনটা খাটবে: বৈধ কাস্টম কোট থাকলে সেটা, না থাকলে এজেন্সির ডিফল্ট দাম, সেটাও না থাকলে পাবলিক দাম। কোটের দাম পাবলিক দামের চেয়ে বেশি হলে সিস্টেম সেলারকে সতর্ক করবে।

WhatsApp কোটের ধাপ

1. এজেন্সি পোর্টালে হোটেল, তারিখ, ধরন আর সংখ্যা দেয়। সিস্টেম খালি রুম আর পাবলিক দাম দেখায়, সাথে **"WhatsApp-এ দাম জিজ্ঞেস করুন"** বাটন।
2. বাটনে ক্লিক করলে সিস্টেম একটা **দামের অনুরোধ** তৈরি করে (রেফারেন্স যেমন `Q-2026-0142`), আর WhatsApp খোলে আগে থেকে লেখা মেসেজসহ।
3. সেলার/ম্যানেজার WhatsApp-এ কথা বলে। তারপর অ্যাডমিন প্যানেলে ওই রেফারেন্স খুলে দাম আর মেয়াদ বসায় (যেমন ৩৫০ SAR, ৪৮ ঘণ্টা)।
4. এজেন্সি পোর্টালে "আপনার দাম: ৩৫০ SAR" দেখে নিজেই বুক করে। অথবা ম্যানেজার নিজেই এজেন্সির পক্ষে বুকিং বানিয়ে দেয়।
5. বুকিং-এ দামটা **স্থির করে সেভ** হয়। পরে দাম বদলালেও এই বুকিং-এর দাম বদলাবে না।

```
WhatsApp লিংক (click-to-chat, কোনো API খরচ নেই):
https://wa.me/9665XXXXXXXX?text=<URL-encoded মেসেজ>

আগে থেকে লেখা মেসেজ:
আসসালামু আলাইকুম। দামের অনুরোধ: Q-2026-0142
এজেন্সি: আল-নূর ট্রাভেলস
হোটেল: MKK-GRAND · ১–১০ জুন ২০২৭ (৯ রাত)
ধরন: গ্রুপ · পুরুষ ১৮, মহিলা ১২ (মোট ৩০)
```

কোটের স্ট্যাটাস: `REQUESTED` → `OFFERED` → `ACCEPTED` (বুক হয়েছে), অথবা `EXPIRED` / `DECLINED`। কোট নির্দিষ্ট হোটেল আর তারিখের জন্য। এজেন্সি হাজির সংখ্যা বাড়ালে বা তারিখ বদলালে নতুন কোট লাগবে। পরে চাইলে WhatsApp Business API যোগ করে "দাম তৈরি" বা "বুকিং কনফার্ম" নোটিফিকেশন স্বয়ংক্রিয়ভাবে পাঠানো যাবে।

- **প্রতি রাতের দাম আলাদা লাইনে স্থির হবে,** কারণ এক থাকার সময় দুই সিজনে পড়তে পারে (যেমন রমজানের শেষ আর তার পরের দিনগুলো)।

- টাকা integer **হালালায়** (১ SAR = ১০০ হালালা), float কখনো নয়। দামের সাথে VAT ধরা আছে কিনা সেটাও সেভ হবে।
- **বুকারকে দেখানো দাম** = প্রতি রাতে (সিট নেওয়া হাজি × জনপ্রতি দাম) + এক্সট্রা বেড, দাম জনপ্রতি হলে। খালি সিটের দাম ধরা হবে না। আসল রুম বাছার পরে দাম আবার হিসাব হবে।
- **একটা কোট একটা বুকিং-এই ব্যবহার করা যাবে।** বুকিং-এর হোটেল, তারিখ, ধরন আর সংখ্যা কোটের সাথে মিলতে হবে।

> **কোট রুম আটকায় না।** ৪৮ ঘণ্টার মধ্যে রুম শেষ হয়ে গেলে কোট থাকলেও বুক হবে না। ক্লায়েন্ট আর এজেন্সি দুজনকেই এটা স্পষ্ট জানাতে হবে, আর পোর্টালে কোটের পাশে লেখা থাকবে। কোটের সাথে রুম আটকে রাখা সম্ভব, কিন্তু তাতে কেনা হবে না এমন রুমও আটকে থাকার ঝুঁকি।

৯. রুম অ্যালোকেশন অ্যালগরিদম

৯.১ একটা রুমে কয়টা সিট ব্যবহারযোগ্য

```
রাতের ক্যাপাসিটি   = base config বা সিজনাল override (ক্লায়েন্টের ক্যালেন্ডার ইঞ্জিন থেকে)
রাতের খালি সিট    = রাতের ক্যাপাসিটি + এক্সট্রা বেড − ওই রাতে বুক করা সিট
ব্যবহারযোগ্য সিট  = পুরো থাকার সময়ের সব রাতের মধ্যে সবচেয়ে কম খালি সিট
```

থাকার সময়ের কোনো রাতে রুমটা ব্লক (রক্ষণাবেক্ষণ ইত্যাদি) থাকলে, অথবা অন্য মালিক বা অন্য লিঙ্গের হাজি থাকলে, রুমটা পুরোপুরি বাদ।

৯.২ গ্রুপ বুকিং-এর ধাপ

1. **কোটা চেক (এজেন্সি হলে, লকসহ: সেকশন ১০)।** থাকার প্রতিটা রাতে: এজেন্সির আগের বুকিং (HELD সহ) + নতুন পুরুষ + নতুন মহিলা ≤ কোটা। পার হলে বুকিং হবে না, আর সর্বোচ্চ কতজন সম্ভব সেটা দেখাবে।
2. **পুরুষ আর মহিলার জন্য আলাদাভাবে ধাপ ৩–৬ চলবে।** ১০ বছরের কম বয়সী শিশু যে পাশে থাকবে (ডিফল্ট মায়ের সাথে), সেই পাশের সংখ্যায় যোগ হবে। কোলের বাচ্চা সিট নেয় না।
3. **নিজের রুম আগে।** শর্ত তিনটা: একই এজেন্সি, একই লিঙ্গ, আর আগের বুকিং-এর **চেক-ইন ও চেক-আউট হুবহু এক**। এমন রুমের খালি সিট আগে ভরবে। লক্ষ্য রুমগুলো পুরো ভরে ফেলা: যে রুমগুলো একদম পূর্ণ হয়ে যায় সেগুলো আগে।
4. **বাকিদের জন্য নতুন রুম।** শুধু পুরো থাকার সময় সম্পূর্ণ খালি রুম বিবেচনায় আসবে। গ্রুপের আকার দেখে অ্যাডমিনের রুম-মিক্স নিয়ম প্রয়োগ হবে। তারপর সব সম্ভাব্য কম্বিনেশন যাচাই, এই ক্রমে বিচার করে:
   a. রুম-মিক্সের সীমা মানে কিনা,
   b. সবচেয়ে কম সিট খালি থাকে,
   c. সবচেয়ে কম রুম লাগে,
   d. সবচেয়ে কম খরচ,
   e. সব সমান হলে বড় রুম আগে।
5. **এক্সট্রা বেড অপশন।** সিট কম পড়লে, অথবা এক্সট্রা বেড দিলে অন্তত একটা রুম কম লাগে, তখন এক্সট্রা বেডসহ একটা অপশনও তৈরি হবে (সেকশন ১১)।

6. **নির্দিষ্ট রুম বাছাই।** একই বা কাছাকাছি ফ্লোর, এজেন্সির আগের রুমগুলোর কাছাকাছি, আর একই লিঙ্গের রুমগুলো পাশাপাশি। রুম বাছার পরে দাম আবার হিসাব হবে।
7. **অপশন দেখানো ও হোল্ড।** পুরুষ আর মহিলার ফলাফল মিলিয়ে ২–৩টা পূর্ণ অপশন, দামসহ। বুকার একটা বাছলে **সেই মুহূর্তেই** রুম লক করে আবার যাচাই করে HELD সেভ হবে (সেকশন ১০)। কনফার্মের সময় শুধু হোল্ডের মেয়াদ দেখা হবে।

৯.৩ ফ্যামিলি বুকিং-এর ধাপ

1. লিঙ্গ দেখা হবে না, রুম-মিক্স নিয়ম খাটবে না।
2. শুধু সম্পূর্ণ খালি রুম। ফ্যামিলির রুম অন্য কোনো বুকিং-এর সাথে ভাগ হবে না, একই এজেন্সির হলেও না।
3. সিস্টেম মোট সংখ্যা ধরে সাজেস্ট করবে: সবচেয়ে কম খালি সিট, তারপর সবচেয়ে কম রুম। ইউজার চাইলে রুম টাইপ আর সংখ্যা নিজে বদলাতে পারবে (রুম হিসেবে বুকিং)।
4. এক্সট্রা বেডের নিয়ম গ্রুপের মতোই।
5. সাধারণ ইউজারের জন্য প্রতি বুকিং-এ সর্বোচ্চ কয়টা রুম, সেটা অ্যাডমিন ঠিক করবে।

৯.৪ রুম-মিক্স ও অপশন বের করার কোড

সংস্করণ ৫-এ সব কম্বিনেশন একে একে যাচাই (brute force) করা হতো, যেটা ১৫০–৩০০ জনের হজ গ্রুপে অচল (১৫০ জনে প্রায় ২ সেকেন্ড, ৩০০ জনে ৪.৭ কোটি কম্বিনেশন)। সংস্করণ ৬-এ এর জায়গায় DP এসেছিল, কিন্তু সেটাতে দুইটা ভুল ছিল:

- **এক্সট্রা বেডের অপশন হারিয়ে যেত।** প্রতিটা সিট-সংখ্যায় শুধু সবচেয়ে কম রুম আর কম খরচের ৩টা কম্বিনেশন রাখা হতো, এক্সট্রা বেডের জায়গা আছে কিনা না দেখে। এক্সট্রা বেড শুধু কিছু রুম টাইপে চলে, তাই যে কম্বিনেশনে জায়গা ছিল সেটা বাদ পড়ে যেতে পারত।
- **হজে ৬ সিটের রুম সীমা এড়িয়ে যেত।** সীমা লেখা ছিল ঠিক "৫ সিট" ধরে। হজে একটা Quad ৬ সিট হলে তার কোনো সীমা থাকত না।

সংস্করণ ৬-এর টেস্ট এগুলো ধরতে পারেনি, কারণ তুলনার "সঠিক উত্তর" পদ্ধতিতেও একই সরলীকরণ ছিল। এবার তুলনা করা হয়েছে **আসল রুমগুলোর প্রতিটা সম্ভাব্য সেট** যাচাই করে: প্রতিটা রুমের নিজের দাম আর এক্সট্রা বেডের জায়গা, ৬ সিটের রুম, আর "৫ বা বেশি সিট" সীমা ধরে। পুরোনো DP ৫,০০০ কেসের ১,৩৮৩টায় ভুল উত্তর দিয়েছিল। নতুন DP ১০,০০০ কেসে একটাতেও ভুল করেনি (এর মধ্যে ১,২৩৬টায় এক্সট্রা বেডের অপশন ছিল, ৫,৫৪৭টায় সীমার আওতায় ৬ সিটের রুম)।

যা বদলেছে:

- **এক্সট্রা বেডের জায়গাও অবস্থার অংশ।** প্রতিটা "t-টা সিট" অবস্থা এখন জায়গা অনুযায়ী আলাদা: e = min(এক্সট্রা বেডের জায়গা, বুকিং-এ সর্বোচ্চ এক্সট্রা বেড), মানে শুধু ০, ১ বা ২। প্রতিটা (t, e)-তে সেরা কম্বিনেশন রাখা হয়, তাই জায়গাওয়ালা কম্বিনেশন আর হারায় না।
- **সীমা এখন "৫ বা তার বেশি সিট"**, হজে ৬ সিট হয়ে যাওয়া রুমসহ। সীমিত রুমের সংখ্যাও অবস্থায় গোনা হয়। সীমিত সাইজগুলো আগে হিসাব হয়, তারপর এই হিসাব গুটিয়ে ফেলা হয়, যাতে বাকি হিসাব হালকা থাকে।
- **একই সাইজের রুমের ভেতরে ছোট একটা knapsack:** n-টা রুম নিলে, নির্দিষ্ট এক্সট্রা-জায়গাসহ সবচেয়ে কম দাম কত। শুধু "সস্তা রুম আগে" নেওয়া আর যথেষ্ট নয়, কারণ সস্তা রুমে হয়তো এক্সট্রা বেডের জায়গা নেই।

হাজি (এক লিঙ্গে)	সংস্করণ ৬ (ভুল ছিল)	সংস্করণ ৭ (সঠিক)
৩০	প্রায় ১ ms	প্রায় ৩ ms
১৫০	প্রায় ১৯ ms	প্রায় ৯০ ms
৩০০	প্রায় ৫৬ ms	প্রায় ২৪০ ms

(Node.js 22, ৪২০টা রুমের হোটেলে মাপা।) নতুন কোড কিছুটা ধীর, কারণ এখন সঠিক। একটা বুকিং খোঁজায় এটা যথেষ্ট দ্রুত। দরকার হলে প্রতিটা কম্বিনেশনের `mix` কপি না করে শুধু আগের অবস্থার রেফারেন্স (back-pointer) রেখে আরও দ্রুত করা যায়।

```
type Room = { id: string; seats: number; price: number; extraPerRoom: number }; // seats
= পুরো থাকার সময়ের ব্যবহারযোগ্য সিট
type CapRule = { minSeats: number; maxRooms: number } | null;                 // যেমন "৫
বা বেশি সিটের রুম সর্বোচ্চ ২টা"
type Policy = { cap: CapRule; extra: { maxPerBooking: number; price: number } };
type Cand = { rooms: number; cost: number; mix: Record<number, number> };
type Option = Cand & { extraBeds: number; emptySeats: number };

const K = 2;                                            // প্রতিটা অবস্থায় সেরা ২টা কম্বিনেশন
const better = (a: Cand, b: Cand) => a.rooms - b.rooms || a.cost - b.cost;
const keep = (list: Cand[]) => list.sort(better).slice(0, K);

// একটা সাইজের ভেতরে: n-টা রুম নিলে, এক্সট্রা-বেডের জায়গা e (E-তে সীমিত) হলে সবচেয়ে কম দাম
function bestWithinSize(list: Room[], maxN: number, E: number): number[][] {
  const f = Array.from({ length: maxN + 1 }, () => Array(E + 1).fill(Infinity));
  f[0][0] = 0;
  for (const r of list)
    for (let n = maxN - 1; n >= 0; n--)
      for (let e = 0; e <= E; e++) if (f[n][e] < Infinity) {
        const e2 = Math.min(E, e + r.extraPerRoom);
        f[n + 1][e2] = Math.min(f[n + 1][e2], f[n][e] + r.price);
      }
  return f;
}

export function suggestOptions(need: number, rooms: Room[], p: Policy): Option[] {
  const E = p.extra.maxPerBooking;
  const bySize = new Map<number, Room[]>();
  for (const r of rooms) bySize.set(r.seats, [...(bySize.get(r.seats) ?? []), r]);
  const capped = (s: number) => p.cap !== null && s >= p.cap.minSeats;
  // সীমিত সাইজগুলো আগে; সেগুলো শেষ হলে c-মাত্রা গুটিয়ে ফেলা যায়
  const sizes = [...bySize.keys()].sort((a, b) => Number(capped(b)) -
Number(capped(a)));
  const limit = need + Math.max(0, ...sizes);
  const C = p.cap?.maxRooms ?? 0;

  // অবস্থা: [t সিট][e এক্সট্রা-জায়গা][c সীমিত রুম] → সেরা K-টা
  let dp: Cand[][][][] = Array.from({ length: limit + 1 }, () =>
    Array.from({ length: E + 1 }, () => Array.from({ length: C + 1 }, () => [] as
```

```
Cand[])));
  dp[0][0][0] = [{ rooms: 0, cost: 0, mix: {} }];

  for (const s of sizes) {
    const isCapped = capped(s);
    const maxN = Math.min(bySize.get(s)!.length, Math.ceil(limit / s), isCapped ? C :
Infinity);
    const f = bestWithinSize(bySize.get(s)!, maxN, E);
    const next = dp.map(a => a.map(b => b.map(c => [...c])));     // এই সাইজের ০টা রুম
    for (let t = 0; t <= limit; t++) for (let e = 0; e <= E; e++) for (let c = 0; c <=
C; c++)
      for (const cand of dp[t][e][c])
        for (let n = 1; n <= maxN && t + n * s <= limit; n++) {
          const c2 = isCapped ? c + n : c;
          if (c2 > C) break;
          for (let es = 0; es <= E; es++) if (f[n][es] < Infinity)
            next[t + n * s][Math.min(E, e + es)][c2].push({ rooms: cand.rooms + n,
              cost: cand.cost + f[n][es], mix: { ...cand.mix, [s]: n } });
        }
    dp = next.map(a => a.map(b => b.map(keep)));
    if (isCapped && !sizes.slice(sizes.indexOf(s) + 1).some(capped))   // শেষ সীমিত সাইজ
      dp = dp.map(a => a.map(b => [keep(b.flat()), ...b.slice(1).map(() => [])]));
  }

  const plain: Option[] = [], withExtra: Option[] = [];
  for (let t = 0; t <= limit; t++) for (let e = 0; e <= E; e++) for (let c = 0; c <= C;
c++)
    for (const cand of dp[t][e][c]) {
      if (cand.rooms === 0) continue;
      if (t >= need) plain.push({ ...cand, extraBeds: 0, emptySeats: t - need });
      else if (need - t <= e)                                  // e আগেই maxPerBooking-এ
সীমিত
        withExtra.push({ ...cand, extraBeds: need - t, emptySeats: 0, cost: cand.cost +
(need - t) * p.extra.price });
    }
  plain.sort((a, b) => a.emptySeats - b.emptySeats || a.rooms - b.rooms || a.cost -
b.cost);
  withExtra.sort((a, b) => a.extraBeds - b.extraBeds || a.rooms - b.rooms || a.cost -
b.cost);
  const options = plain.slice(0, 2);
  // এক্সট্রা বেড অপশন: শুধু সিট কম পড়লে, অথবা এতে অন্তত একটা রুম কম লাগলে
  if (withExtra[0] && (!plain[0] || withExtra[0].rooms < plain[0].rooms))
options.push(withExtra[0]);
  return options;
}
```

টেস্টে যা রাখতে হবে: ছোট র‍্যান্ডম ইনপুটে (৩–১২টা রুম) আসল রুমগুলোর প্রতিটা সম্ভাব্য সেট যাচাই করে "সঠিক উত্তর" বের করা, আর DP-র পুরো অপশন তালিকার সাথে মেলানো, এক্সট্রা বেডের অপশনসহ। প্রতিটা রুমের দাম আর এক্সট্রা বেডের জায়গা আলাদা র‍্যান্ডম হবে, ৬ সিটের রুম থাকবে, আর সীমা কখনো থাকবে, কখনো থাকবে না।

খরচ শুধু সাজানোর জন্য, দেখানো দাম আলাদা। এখানে `cost` হলো রুমগুলোর দাম যোগ করে পাওয়া একটা মাপ, যেটা দিয়ে শুধু অপশন সাজানো হয়। দাম জনপ্রতি হলে বুকারকে দেখানো দাম হবে প্রতি রাতে (সিট নেওয়া হাজি × জনপ্রতি দাম) + এক্সট্রা বেড। খালি সিটের দাম কেউ দেবে না, হোটেল সেটা বহন করবে (সেকশন ২৩)। ধাপ ৬-এ আসল রুম বাছার পরে দাম আবার হিসাব হবে, কারণ কাছাকাছি ফ্লোরের জন্য একই সাইজের একটু দামি রুম আসতে পারে।

এক্সট্রা বেডের অপশন সাজানো: খরচ দিয়ে নয়, কম এক্সট্রা বেড আগে। এক্সট্রা বেড একটা রুমের চেয়ে সস্তা, তাই খরচ দিয়ে সাজালে সিস্টেম বেশি মানুষকে এক্সট্রা বেডে বসানোর দিকে ঝুঁকত। কম এক্সট্রা বেড মানে কম ভিড়, আর হোটেল রুম বেশি বিক্রি করে।

১০. একসাথে অনেক বুকিং: লক, হোল্ড ও কোটা

দুইজন একই সময়ে একই রুম চাইলে ডাবল-বুকিং যাতে না হয়, সেজন্য এই নিয়মগুলো সবচেয়ে গুরুত্বপূর্ণ। ডাটাবেস constraint (সেকশন ১৬) দুই আলাদা মালিককে এক রুমে আটকায়। কিন্তু **একই মালিকের দুইটা হোল্ড একসাথে এসে একটা রুম ক্যাপাসিটির বেশি ভরে ফেলা** constraint আটকায় না। সেটা আটকাবে নিচের লক।

হোল্ড তৈরির transaction

অপশন বাছার মুহূর্তেই সব যাচাই আর লক হবে, কারণ HELD অবস্থাতেই সিট আটকে যায়। সব কাজ একটা transaction-এ, এই ক্রমে:

```
BEGIN
 ১. কোটা লক          pg_advisory_xact_lock(hash(agency_id, hotel_id))
                     একই এজেন্সির দুইটা বুকিং একসাথে কোটা পাস করতে পারবে না
 ২. রুম লক           SELECT ... FROM room WHERE id IN (...) ORDER BY id FOR UPDATE
 ৩. পুরোনো হোল্ড সরানো   এই রুমগুলোর মেয়াদোত্তীর্ণ HELD/REQUESTED → EXPIRED
 ৪. আবার যাচাই       প্রতি রাতে: guest_stay গুনে খালি সিট, মালিক, লিঙ্গ, ব্লক, কোটা
 ৫. HELD সেভ          booking, booking_room, booking_guest, guest_stay
                     hold_expires_at = now() + ১০ মিনিট
COMMIT
```

- **লকের ক্রম সবসময় একই:** কোটা → রুম (আইডির ক্রমে) → বুকিং-এর row। রুম লক নেওয়ার পর ওই রুমের হোল্ড আর কোনো transaction ছুঁতে পারে না, তাই পুরোনো হোল্ড সরানো নিরাপদ। deadlock হয় না।
- **ধাপ ৪-এ কিছু না মিললে** (যেমন কেউ এক মুহূর্ত আগে নিয়ে নিয়েছে) পুরো transaction বাতিল হবে, আর সিস্টেম নতুন অপশন দেখাবে।
- **কনফার্মের সময়** আর রুম লক বা যাচাই লাগবে না। শুধু বুকিং-এর row লক করে দেখবে হোল্ডের মেয়াদ আছে কিনা, তারপর CONFIRMED।
- সাধারণ ইউজারের `REQUESTED` বুকিংও একইভাবে, শুধু মেয়াদ বেশি (যেমন ২৪ ঘণ্টা)।
- **প্রতিটা হোল্ডের জন্য আলাদা জব নেই।** transaction-এর ভেতরে সরানো (ধাপ ৩) আর প্রতি মিনিটের sweep জব, এই দুটোই যথেষ্ট।

> **pg-boss জব একই transaction-এ: আগে পরীক্ষা করতে হবে।** pg-boss ডিফল্টে নিজের connection pool ব্যবহার করে। রিমাইন্ডারের মতো জব বুকিং-এর transaction-এর ভেতরে সেভ করতে হলে pg-boss-কে একটা ছোট adapter দিতে হয় (তার `db` অপশন), যেটা Drizzle transaction-এর connection-এ SQL চালায়। ব্যবহার করা ড্রাইভারের সাথে এটা কাজ করে কিনা, Phase 2a-র শুরুতে একটা ছোট পরীক্ষা (spike) করে নিশ্চিত হতে হবে।

মেয়াদোত্তীর্ণ হোল্ড কেন সমস্যা

ডাটাবেস constraint-এর `WHERE status IN ('HELD', ...)` অংশে `now()` ব্যবহার করা যায় না। তাই কারও হোল্ডের মেয়াদ শেষ হলেও স্ট্যাটাস না বদলানো পর্যন্ত সেটা রুম আটকে রাখে। দুইভাবে সরানো হবে:

1. প্রতিটা বুকিং-এর transaction নিজে, যে রুমগুলো নিতে চায় তাদের পুরোনো হোল্ড আগে সরিয়ে নেবে (উপরের ধাপ ৩)।
2. ব্যাকআপ হিসেবে pg-boss-এর একটা জব প্রতি মিনিটে সব মেয়াদোত্তীর্ণ হোল্ড সরাবে।

কোটা

- কোটা হবে **নির্দিষ্ট সংখ্যা**, প্রতি হোটেলে, প্রতি সিজনে, এক রাতের হিসাবে। যেমন হজ সিজনে ৩০ জন।
- আগের সংস্করণের শতাংশ (%) বাদ, কারণ ক্যাপাসিটি সিজনভেদে বদলায় আর "কিসের ৩০%" নিয়ে বিভ্রান্তি হয়।
- গোনা হবে HELD, REQUESTED আর এক্সট্রা বেডের হাজিসহ।

ইনভেন্টরি বদলালে কনফার্মড বুকিং ভাঙতে পারে

বুকিং আসার পরে Phase 1-এর কিছু কাজ কনফার্মড বুকিং-এর সাথে সংঘর্ষ করতে পারে:

- রুম বা ফ্লোর বন্ধ করা, অথবা রুম আবার চালু করা
- রক্ষণাবেক্ষণ বা অপারেশনাল ব্লক দেওয়া
- সিজনাল ক্যাপাসিটি কমানো, বা ক্যাপাসিটি পিরিয়ড মোছা

নিয়ম: এই প্রতিটা কাজের আগে সংশ্লিষ্ট রুম আর তারিখের বুকিং চেক হবে। সংঘর্ষ থাকলে কাজটা আটকে যাবে, আর কোন কোন বুকিং-এর সাথে সংঘর্ষ সেই তালিকা দেখাবে। ম্যানেজার আগে হাজিদের অন্য রুমে সরাবে (সেকশন ১২), তারপর কাজটা করবে। এই কাজগুলোও একই রুম লক নেবে, যাতে বুকিং-এর সাথে একসাথে চলতে না পারে।

> Phase 1-এর কিছু সার্ভিসে (ফ্লোর বন্ধ করা, রুম আবার চালু করা, ক্যাপাসিটি পিরিয়ড মোছা) এখন লক নেই। বুকিং আসার পর এগুলো সত্যিকারের ওভারবুকিং-এর ঝুঁকি, তাই Phase 2a-তেই ঠিক করতে হবে।

সাধারণ ইউজারের অপব্যবহার ঠেকানো

- একজন ইউজারের একসাথে সর্বোচ্চ ১টা খোলা HELD বা REQUESTED।
- সাইন-আপে ফোন OTP, সন্দেহজনক হলে captcha।
- REQUESTED-এর মেয়াদ (যেমন ২৪ ঘণ্টা)। ম্যানেজার নিশ্চিত না করলে নিজে থেকে EXPIRED।

১১. এক্সট্রা বেড

১. সিট কম পড়লে

হাজি ২০ জন, খালি সিট ১৯টা। বুকিং ফিরিয়ে না দিয়ে সিস্টেম ১টা এক্সট্রা বেড সাজেস্ট করবে।

২. ১–২ জনের জন্য পুরো রুম লাগলে

শেষ ১ জনের জন্য একটা দামি ৪ বা ৫ জনের রুম নিতে হচ্ছে। বুকার বেছে নেবে: নতুন রুম, নাকি নিজের রুমে এক্সট্রা বেড।

- এক্সট্রা বেড শুধু **এই বুকারের নিজের রুমে**। গ্রুপে একই লিঙ্গের রুমে।

- প্রতি রুমে আর প্রতি বুকিং-এ সর্বোচ্চ কয়টা, কোন রুম টাইপে, দাম কত: সব অ্যাডমিনের সেটিং (সেকশন ১৫)।
- এক্সট্রা বেডের হাজিও এজেন্সির **কোটায় গোনা হবে।**
- রিসিপ্ট আর ইনভয়েসে আলাদা লাইনে। কনফার্ম হলে হোটেল স্টাফের জন্য কাজ: "রুম ৪১২-তে ১৫ তারিখের আগে ১টা এক্সট্রা বেড বসাতে হবে।"
- চেক-ইনে এক্সট্রা বেডের slot-এও একজন হাজির তথ্য দিতে হবে।

বিষয়	সিজনাল ক্যাপাসিটি (ক্লায়েন্টের সিস্টেমে আছে)	এক্সট্রা বেড (নতুন)
কে ঠিক করে	হোটেল, পুরো সিজনের জন্য	বুকার, নির্দিষ্ট বুকিং-এর সময় চায়
কার জন্য	সবার জন্য একই	শুধু ওই বুকিং-এর জন্য
চার্জ	রুমের দামের মধ্যে	আলাদা চার্জ
কোথায় থাকে	`room_capacity_override`	`booking_room.extra_beds`

১২. বুকিং বদল, বাতিল ও চেক-ইন

"বুক → চেক-ইন → চেক-আউট" ছাড়াও বাস্তবে অনেক কিছু বদলায়। Phase 2-এর সবচেয়ে কঠিন অংশ সাধারণত এগুলোই, বিশেষ করে বুকিং বদল।

ঘটনা	কী হবে	নিয়ম
হাজি যোগ	একই বুকিং-এ নতুন slot	প্রথমে বুকিং-এর নিজের রুমের খালি সিট, তারপর অ্যালোকেশন ইঞ্জিন। কোটা আবার চেক। দাম বুকিং-এর কোট বা রেট অনুযায়ী।
হাজি বাদ	slot `REMOVED`	রুমে খালি সিট তৈরি হয়। গ্রুপে সেটা শুধু একই এজেন্সির একই তারিখের বুকিং পাবে। চার্জ বাতিল নীতি অনুযায়ী।
তারিখ বদল	পুরো বুকিং নতুন তারিখে	কার্যত নতুন অ্যালোকেশন। নতুন তারিখে রুম না পেলে আগের বুকিং যেমন ছিল তেমনই থাকবে (হয় পুরোটা, নয় কিছুই না)। দাম প্রতি রাতের হিসাবে নতুন করে।
রুম বদল	হাজিকে অন্য রুমে	রক্ষণাবেক্ষণ, অভিযোগ বা ইনভেন্টরি বদলের জন্য। একই মালিক আর লিঙ্গের নিয়ম। কোন তারিখ থেকে বদল, সেটা রেকর্ড।
বাতিল	পুরো বুকিং `CANCELLED`	কত দিন আগে বাতিলে কত চার্জ, সেই নীতি ক্লায়েন্ট ঠিক করবে। ব্যবহৃত কোট আর কাজে লাগবে না।
No-show	হাজি আসেনি	চেক-ইনের দিন শেষে রিসেপশন `NO_SHOW` করবে। সিট ছেড়ে দেওয়া হবে কিনা নীতি অনুযায়ী।
আগে চেক-আউট	হাজি আগে চলে গেল	বাকি রাতের সিট খালি হবে। টাকা ফেরত কিনা নীতি অনুযায়ী।
চেক-ইন	প্রতিটা হাজি আলাদা	২০ জন সাধারণত একসাথে আসে না। প্রতিজন আলাদা `CHECKED_IN`, তার আগে তার পুরো তথ্য বাধ্যতামূলক। প্রথম জন এলে বুকিং `IN_HOUSE`।

- প্রতিটা বদল সেকশন ১০-এর একই লক নিয়মে চলবে।
- ডাটাবেসে রুম বদল বা আগে চেক-আউট মানে `guest_stay`-এর একটা row ছোট করা, আর দরকার হলে নতুন একটা যোগ করা (সেকশন ১৬)।
- প্রতিটা বদল `booking_change` টেবিলে থাকবে: কী ছিল, কী হলো, কে করল, কখন। পাসপোর্টের তথ্য বাদ দিয়ে।
- এজেন্সি কোন বদল নিজে করতে পারবে আর কোনটায় ম্যানেজারের অনুমোদন লাগবে, সেটা ক্লায়েন্ট ঠিক করবে। আপাতত ধরে নেওয়া: হাজি যোগ এজেন্সি নিজে করবে, বাকি সব ম্যানেজারের অনুমোদনে।

১৩. পাসপোর্টের ছবি থেকে স্বয়ংক্রিয় তথ্য

এজেন্সি পোর্টাল আর হোটেল প্যানেলে একই স্ক্রিন। লক্ষ্য: হাতে টাইপ প্রায় শূন্য।

ব্যবহারকারী যা দেখবে

1. বুকিং খুলে **"হাজির তথ্য যোগ"** → একটা বড় ড্রপ-জোন: "পাসপোর্টের ছবি এখানে ছাড়ুন"। একটা বা একসাথে অনেকগুলো ছবি (যেমন ৩০টা) দেওয়া যাবে। মোবাইলে সরাসরি ক্যামেরা দিয়ে তোলা যাবে, রিসেপশনের জন্য সুবিধা।
2. প্রতিটা ছবি কয়েক সেকেন্ডে পড়া হবে। তথ্য ফর্মে বসে যাবে, সন্দেহজনক ফিল্ড হলুদ রঙে চিহ্নিত।
3. অনেক ছবি দিলে সিস্টেম নিজেই খালি slot-এ বসাবে। গ্রুপে পাসপোর্টের লিঙ্গ দেখে পুরুষ বা মহিলার slot-এ।
4. প্রতিটা হাজির জন্য একটা ছোট রিভিউ কার্ড: ছবি পাশে, তথ্য পাশে। "ঠিক আছে, পরের জন" চাপলে পরেরটা।
5. সবশেষে **"কনফার্ম"** চাপলে সব হাজির একটা সারসংক্ষেপ ফর্ম। আরেকবার যাচাই করে সেভ।

পাসপোর্ট থেকে কোন তথ্য

তথ্য	কোথা থেকে	যাচাই
পাসপোর্ট নম্বর	MRZ	check digit দিয়ে
নাম (পদবি, নামের অংশ)	MRZ + ছাপা অংশ	MRZ-এ লম্বা নাম কেটে যেতে পারে, তাই পুরো নাম ছাপা অংশ থেকে
জাতীয়তা, লিঙ্গ	MRZ	গ্রুপে লিঙ্গ slot-এর সাথে মেলানো
জন্মতারিখ, মেয়াদ শেষের তারিখ	MRZ	check digit; ন্যূনতম মেয়াদ (অ্যাডমিন সেটিং)
ইস্যুর তারিখ, জন্মস্থান, ইস্যুকারী কর্তৃপক্ষ	ছাপা অংশ	AI দিয়ে পড়া, রিভিউতে দেখা
ঠিকানা	ছাপা অংশ (থাকলে)	অনেক দেশের পাসপোর্টের ডেটা পেজে ঠিকানা থাকে না। না পেলে হাতে দিতে হবে।
মুখের ছবি	ছবি থেকে কেটে নেওয়া	চেক-ইনে মিলিয়ে দেখার জন্য

পেছনে কীভাবে কাজ করবে

```
ছবি আপলোড → এনক্রিপ্ট করে স্টোরেজে (ক্লায়েন্টের document_asset, SHA-256)
          → pg-boss জব
             ১. নিজের সার্ভারে MRZ পড়া + check digit যাচাই
             ২. বাকি ফিল্ড নিজের সার্ভারের OCR দিয়ে, MRZ-এর সাথে মিলিয়ে
             ৩. MRZ পড়া না গেলে বা ফিল্ড অস্পষ্ট হলে তবেই বাইরের AI vision (fallback)
             ৪. প্রতিটা ফিল্ডে confidence
          → রিভিউ ফর্মে দেখানো → মানুষ নিশ্চিত করলে তবেই pilgrim-এ সেভ
```

- **প্রযুক্তি:** MRZ পড়ার ওপেন-সোর্স লাইব্রেরি আর OCR প্রথমে নিজের সার্ভারে, **আলাদা worker প্রসেসে** (CPU-ভারী)। বাইরের AI vision মডেল (যেমন Claude বা অন্য কোনো vision API) শুধু fallback। বাছাইয়ের আগে বাংলাদেশ, পাকিস্তান, ইন্দোনেশিয়া ইত্যাদি দেশের আসল পাসপোর্টের নমুনায় নির্ভুলতা যাচাই করে নিতে হবে।
- **প্রত্যাশা ঠিক রাখা:** নিজের সার্ভারের OCR MRZ-এ খুব নির্ভরযোগ্য, কিন্তু ছাপা অংশে বাংলা, উর্দু বা আরবি লেখায় দুর্বল। নাম, তারিখ আর নম্বরের মতো দরকারি ফিল্ড সাধারণত ইংরেজিতেও থাকে, কিন্তু ঠিকানার মতো স্থানীয় ভাষার ফিল্ডে AI fallback যতটা ভাবা হচ্ছে তার চেয়ে বেশি লাগবে। তাই AI-এর খরচ আর ডেটা-চুক্তি শুরু থেকেই হিসাবে রাখতে হবে।
- **বাড়তি যাচাই:** একই পাসপোর্ট একই সময়ে দুই বুকিং-এ থাকলে সতর্কতা (খোঁজা হয় `passport_lookup` হ্যাশ দিয়ে; অন্য এজেন্সির বুকিং হলে সতর্কতা শুধু স্টাফ দেখবে, সেকশন ১৬)। মেয়াদ কম হলে সতর্কতা।

পাসপোর্টের তথ্যের সুরক্ষা (PDPL)

- **কে কী দেখেছে সেটাও লগ:** বর্তমান `audit_log` শুধু পরিবর্তন রাখে। পাসপোর্টের তথ্য বা ছবি কে কখন দেখেছে, তার জন্য আলাদা `pii_access_log`।
- **এনক্রিপশন:** ছবি আর পাসপোর্ট নম্বর এনক্রিপ্ট করে রাখা। এনক্রিপশন কী কে রাখবে (ক্লাউডের key management সার্ভিস, নাকি ক্লায়েন্টের নিজের) সেটা ঠিক করতে হবে।
- **কতদিন রাখা হবে:** একটা মেয়াদ (যেমন চেক-আউটের ১২ মাস পরে ছবি মোছা), তারপর pg-boss জব নিজে থেকে মুছবে।
- **বাইরের AI সার্ভিস:** ব্যবহার করলে চুক্তি আর ডেটা কোন দেশে যাচ্ছে, সেটা যাচাই।
- **লগে পাসপোর্টের তথ্য নয়:** `audit_log` আর `booking_change`-এ লেখার আগে PII লুকানো, কারণ এগুলো মোছা যায় না (সেকশন ১৬)।
- **কে দেখতে পারবে:** শুধু অনুমোদিত রোল। ক্যালেন্ডার বা তালিকায় পাসপোর্ট নম্বর লুকানো।

১৪. অকুপেন্সি ক্যালেন্ডার (ম্যানেজার ও অ্যাডমিন)

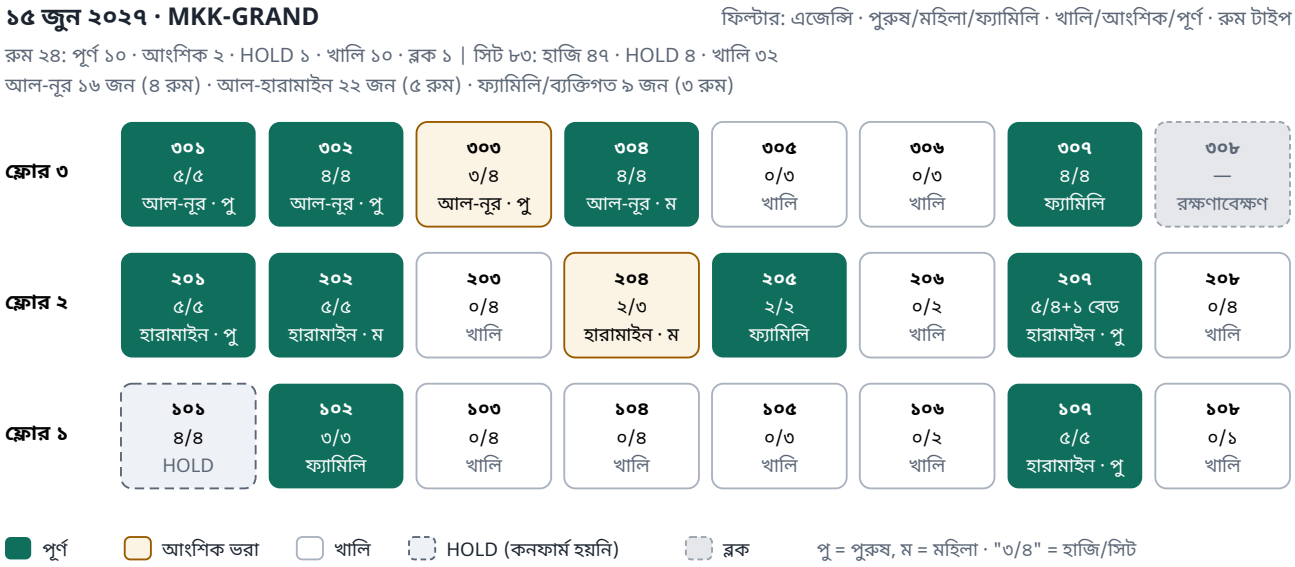
একটা ক্যালেন্ডার, যেখান থেকে যেকোনো দিনে কোন ফ্লোরের কোন রুম বুকড, কোনটা খালি, বুকড রুমে কয়জন হাজি, আর তারা কোন এজেন্সির নাকি ব্যক্তিগত (ফ্যামিলি), সব এক জায়গায় দেখা যাবে।

- **কে দেখবে:** অ্যাডমিন সব হোটেলের, হোটেল ম্যানেজার শুধু নিজের হোটেলের। চাইলে রিসেপশনকে শুধু দেখার অনুমতি দেওয়া যায়।
- **এজেন্সি বা সাধারণ ইউজার** এটা দেখতে পারবে না।

তিনটা ভিউ

ভিউ	কী দেখাবে
বছর	১২ মাসের ছোট ছোট ক্যালেন্ডার একসাথে। প্রতিটা দিন রঙ দিয়ে বোঝাবে কতটা ভরা: সবুজ (৫০%-এর কম), হলুদ (৫০–৯০%), লাল (৯০%-এর বেশি)। হজ আর রমজানের সিজন আলাদা করে চিহ্নিত। কোনো দিনের ওপর মাউস নিলে বুকড আর মোট সিট দেখাবে।
মাস	পুরো মাসের বড় গ্রিড, প্রতিটা দিনের ঘরে: কত % ভরা, খালি রুম ও খালি সিট, ওই দিনের চেক-ইন আর চেক-আউটের সংখ্যা। "এই মাস" আর "পরের মাস" বাটন, আগে-পরে যাওয়া যাবে।
দিন	যেকোনো দিনে ক্লিক করলে ফ্লোরভিত্তিক রুমের ম্যাপ: প্রতিটা ফ্লোর একটা সারি, প্রতিটা রুম একটা টাইল (নিচের ছবি)। উপরে দিনের সারসংক্ষেপ।

দিনের ভিউ কেমন দেখাবে



(ছবির সংখ্যাগুলো উদাহরণ, একে অপরের সাথে মেলানো: ২৪টা রুম, ৮৩টা বিক্রিযোগ্য সিট।)

রুমে ক্লিক করলে যা দেখাবে

```
রুম ৩০৩ · ৪ জনের রুম · ফ্লোর ৩ · পুরুষ
বুকিং B-2027-0088 · আল-নূর ট্রাভেলস (এজেন্সি) · গ্রুপ · ১–১০ জুন
সিট: ৩/৪ ভরা · এক্সট্রা বেড: ০
  ১. Md. Karim            পাসপোর্ট ✓   চেক-ইন ✓
  ২. Abdul Rahim          পাসপোর্ট ✓   চেক-ইন ✓
  ৩. (তথ্য বাকি)          –             –
  ৪. খালি সিট (শুধু আল-নূরের, একই তারিখের বুকিং-এর জন্য)
[বুকিং খুলুন]  [হাজির তথ্য যোগ]
```

দিনের সারসংক্ষেপে যা থাকবে

- মোট রুম: পূর্ণ, আংশিক, HOLD, খালি, ব্লক।
- মোট সিট: কয়জন হাজি, কয়টা HOLD-এ, কয়টা খালি।
- এজেন্সিভিত্তিক হিসাব (কোন এজেন্সির কতজন, কয়টা রুমে), আর ফ্যামিলি/ব্যক্তিগত বুকিং আলাদা।
- আজকের চেক-ইন আর চেক-আউট, আর কতজনের তথ্য এখনো বাকি।
- দিনের তালিকা PDF বা Excel-এ নামানো যাবে, রিসেপশনের জন্য সুবিধা।

কীভাবে বানানো হবে

- **নতুন টেবিল লাগবে না।** সব তথ্য আসবে `booking_room`, `booking_guest` আর `guest_stay` থেকে; সিট গোনা হবে অ্যালোকেশনের মতোই `room_night_occupancy` view দিয়ে, সাথে রুমের ক্যাপাসিটি আর ব্লক ক্লায়েন্টের ক্যালেন্ডার ইঞ্জিন থেকে।
- **দুইটা API:**

```
GET /api/hotels/:id/occupancy?from=2027-01-01&to=2027-12-31
    → প্রতিটা দিনের সারসংক্ষেপ (বছর আর মাসের ভিউয়ের জন্য)

GET /api/hotels/:id/occupancy/2027-06-15
    → ফ্লোর → রুম → বুকিং ও হাজি (দিনের ভিউয়ের জন্য)
```

- **পারফরম্যান্স:** বছরের ভিউতে ৩৬৫ দিন × কয়েকশো রুম। ক্লায়েন্টের ক্যালেন্ডার ইঞ্জিন এরই মধ্যে ৪০০ দিনের হিসাব খুব দ্রুত করে, বুকিং-এর হিসাব সেখানে যোগ হবে। শুরুতে cache ছাড়াই; মেপে দেখে ধীর হলে তখন cache।
- **গোপনীয়তা:** ক্যালেন্ডারে হাজির নাম দেখাবে, কিন্তু পাসপোর্ট নম্বর আর ছবি শুধু অনুমতি থাকা রোল দেখতে পারবে।
- **ক্লায়েন্টের পরিকল্পনার সাথে মিল:** ক্লায়েন্টের Phase 1 UI-তে "ক্যালেন্ডার রিবন ভিউ" এমনিতেই আছে। এই অকুপেন্সি ক্যালেন্ডার সেটার ওপর বুকিং আর হাজির তথ্য বসিয়ে বানানো যাবে।

১৫. অ্যাডমিন সেটিংস

সেটিং	কী ঠিক করে	উদাহরণ
পাবলিক দাম	হোটেল, রুম টাইপ আর সিজন অনুযায়ী	হজ সিজনে জনপ্রতি রাত ৩৮০ SAR
এজেন্সির ডিফল্ট দাম	কোট না থাকলে অনুমোদিত এজেন্সির দাম (ঐচ্ছিক)	পাবলিকের চেয়ে ৫% কম
WhatsApp	কোন নম্বরে যাবে, মেসেজের টেমপ্লেট, কোটের ডিফল্ট মেয়াদ	৪৮ ঘণ্টা
এজেন্সি অনুমোদন	নতুন এজেন্সি অ্যাকাউন্ট অনুমোদন না হলে এজেন্সি দাম বা কোট পাবে না	লাইসেন্স দেখে অনুমোদন
এজেন্সির কোটা	প্রতি হোটেলে, প্রতি সিজনে এক রাতে সর্বোচ্চ কতজন, নির্দিষ্ট সংখ্যায়। ডিফল্ট আর এজেন্সিভিত্তিক।	হজ সিজনে ডিফল্ট ৩০; "আল-নূর" ৪০
সাধারণ ইউজারের সীমা	প্রতি বুকিং-এ সর্বোচ্চ রুম, একসাথে কয়টা খোলা অনুরোধ, অনুরোধের মেয়াদ	৫টা রুম; ১টা অনুরোধ; ২৪ ঘণ্টা
রুম-মিক্স নিয়ম	গ্রুপের আকার অনুযায়ী কোন সাইজের রুম সর্বোচ্চ কয়টা	নিচের টেবিল
এক্সট্রা বেড	কোন রুম টাইপে, প্রতি রুমে ও বুকিং-এ সর্বোচ্চ, দাম, সিজনাল রুমে চলবে কিনা	৩ ও ৪ জনের রুমে ১টা; বুকিং-এ ২টা; ৮০ SAR
হাজির তথ্যের শেষ সময়	চেক-ইনের কত দিন আগে তথ্য দিতে বলা হবে, রিমাইন্ডার	৭ দিন আগে রিমাইন্ডার
পাসপোর্টের ন্যূনতম মেয়াদ	চেক-ইনের তারিখ থেকে কত মাস বাকি থাকতে হবে	৬ মাস
HOLD-এর সময়	অপশন বাছার পর কতক্ষণ সিট আটকে থাকবে	১০ মিনিট
শিশুর বয়সসীমা	কোন বয়সের কম শিশু মা বা বাবার যেকোনো রুমে থাকতে পারবে; ডিফল্টে কার সাথে	১০ বছর; মায়ের সাথে
বাতিল নীতি	কত দিন আগে বাতিল করলে কত চার্জ; কে বাতিল করতে পারবে	ক্লায়েন্ট ঠিক করবে
পাসপোর্টের ছবি রাখার মেয়াদ	চেক-আউটের কত দিন পরে ছবি মোছা হবে	১২ মাস

রুম-মিক্স নিয়মের উদাহরণ (সংখ্যাগুলো ক্লায়েন্ট ঠিক করবে):

গ্রুপের আকার	৫ বা বেশি সিটের রুম সর্বোচ্চ	বাকিরা
১–১০	১টা	৪, ৩, ২, ১ জনের রুম
১১–২০	২টা	৪, ৩, ২, ১ জনের রুম
২১–৩০	৩টা	৪, ৩, ২, ১ জনের রুম
৩১–৪০	৪টা	৪, ৩, ২, ১ জনের রুম

১৬. ডাটা মডেল ও ডাটাবেস নিয়ম

সব টেবিলে টেন্যান্সি

বর্তমান সিস্টেমে প্রতিটা টেবিল `organization_id` দিয়ে আলাদা, আর foreign key কম্পোজিট `(organization_id, id)`। নতুন সব টেবিলেও একই নিয়ম (নিচের তালিকায় আলাদা করে লেখা হয়নি)।

- এজেন্সি আর সাধারণ ইউজার একটা organization-এর (অপারেটরের) ভেতরে থাকবে।
- পাবলিক সাইট যদি ভবিষ্যতে একাধিক অপারেটরের মার্কেটপ্লেস হয়, তাহলে identity, দাম আর পাবলিক সাইটের নকশা বদলাবে। তাই এটা আগেই ক্লায়েন্টের সাথে ঠিক করতে হবে (সেকশন ২৩)।
- এজেন্সি ইউজারের জন্য নতুন `AgencyScope`, সাধারণ ইউজারের জন্য `CustomerScope`। বর্তমান tenant-isolation টেস্ট এগুলোও কভার করবে।

নতুন টেবিল

টেবিল	মূল কলাম
`agency`	নাম, লাইসেন্স, যোগাযোগ, WhatsApp নম্বর, অনুমোদনের স্ট্যাটাস
`agency_user`	এজেন্সির ইউজার। শুধু নিজের এজেন্সির বুকিং দেখবে।
`customer`	সাধারণ ইউজার: নাম, ফোন (OTP যাচাই), ইমেইল
`rate`	hotel_id, room_type_id, সিজন (daterange), price_type (PUBLIC / AGENCY_DEFAULT), amount_halalas, একক (জনপ্রতি বা রুম, প্রতি রাত), vat_included
`price_quote`	ref_no, agency_id, hotel_id, stay, booking_type, পুরুষ/মহিলা/মোট, status, amount_halalas, একক, vat_included, valid_until, offered_by, **used_by_booking_id (unique)**
`agency_quota`	hotel_id, agency_id (খালি = ডিফল্ট), সিজন (daterange), max_persons
`allocation_rule`	hotel_id, group_min, group_max, min_seats (যেমন ৫, মানে "৫ বা বেশি সিট"), max_rooms
`extra_bed_policy`	hotel_id, room_type_id, max_per_room, max_per_booking, amount_halalas, allow_in_override_season
`pilgrim`	**শুধু স্টাফের master পরিচয়।** নাম, পাসপোর্ট নম্বর (এনক্রিপ্টেড), **passport_lookup (unique)**, জাতীয়তা, লিঙ্গ, জন্মতারিখ, পাসপোর্টের মেয়াদ
`booking_guest_detail` (নতুন)	booking_guest_id, বুকার যে তথ্য দিয়েছে (নাম, পাসপোর্ট নম্বর, জন্মতারিখ, ঠিকানা ইত্যাদি, এনক্রিপ্টেড), passport_lookup (unique নয়), entered_by। প্রতিটা বুকারের নিজের কপি।
`passport_scan`	document_asset_id, pilgrim_id, পড়া তথ্য (JSON), confidence, কোন পদ্ধতিতে পড়া (MRZ / OCR / AI), status, reviewed_by, delete_after
`pii_access_log`	কে, কখন, কোন হাজির কোন তথ্য বা ছবি দেখেছে
`booking`	booker_type (AGENCY / INDIVIDUAL), agency_id বা customer_id, created_by (ম্যানেজার হলে), booking_type (FAMILY / GROUP), stay, পুরুষ/মহিলা/মোট, price_source, quote_id, status, hold_expires_at, receipt_no, **receipt_token**, শিশুর সংখ্যা (১০ বছরের কম), কোলের বাচ্চার সংখ্যা, child_side (শিশু কার সাথে)
`booking_room`	booking_id, room_id, owner_agency_id, owner_booking_id, owner_key (generated), occupancy (MALE / FEMALE / FAMILY), stay, status, extra_beds। এটা রুমের **দাবি**: কে, কোন ধরনের, কোন তারিখে। **প্রতিটা বুকিং-এর নিজের row**, একই এজেন্সির দুই বুকিং একই রুম ভাগ করলেও। সিট গোনা হয় `guest_stay` থেকে।
`booking_guest`	booking_id, slot_no, gender (ফ্যামিলিতে খালি), **age_group** (ADULT / CHILD / INFANT; জন্মতারিখ থেকে চেক-ইনের দিনে হিসাব, তথ্য আসার আগে বুকারের দেওয়া), is_extra_bed, pilgrim_id (স্টাফ master-এর সাথে মেলালে), status, checked_in_at, checked_out_at

`guest_stay` (নতুন)	booking_guest_id, booking_room_id, nights (daterange), **occupies_seat**। কোন হাজি কোন রাতগুলোতে কোন রুমে।
`booking_price_line`	booking_id, রাত (date), কী বাবদ (রুম / জন / এক্সট্রা বেড), **quantity**, **unit_amount_halalas**, amount_halalas, vat_included
`booking_change`	booking_id, ধরন (যোগ / বাদ / তারিখ / রুম বদল / বাতিল), আগে ও পরে (JSON, **PII বাদ দিয়ে**), কে, কখন
`hotel_task`	hotel_id, room_id, কাজের ধরন (এক্সট্রা বেড বসানো/সরানো), কবের মধ্যে, স্ট্যাটাস

টাকা: সবসময় integer হালালায় (১ SAR = ১০০ হালালা), float কখনো নয়। দামের সাথে VAT ধরা আছে কিনা সেটাও সেভ হবে, যদিও ইনভয়েস Phase 3-এ। `booking_price_line`-এ quantity আর একক দাম আলাদা, কারণ হাজি যোগ হলে বা আগে চলে গেলে প্রতি রাতের মাথাগুনতি বদলায়।

কোন হাজি কোন রাতে কোন রুমে: `guest_stay`

সেকশন ১২-এর রুম বদল ("৬ তারিখ থেকে অন্য রুমে"), আগে চেক-আউট ("৭ তারিখ থেকে সিট খালি") আর no-show-তে সিট ছাড়া, এগুলো আগের মডেলে রাখা যেত না। `booking_room`-এ একটাই তারিখ-রেঞ্জ আর নির্দিষ্ট সিট সংখ্যা ছিল, আর প্রতিটা হাজি একটাই রুমের সাথে যুক্ত ছিল। এখন:

```
guest_stay(booking_guest_id, booking_room_id, nights daterange, occupies_seat bool)

-- একজন হাজি এক রাতে একটাই বেডে
EXCLUDE USING gist (booking_guest_id WITH =, nights WITH &&)

-- রুম A-তে ১–৫ রাত, রুম B-তে ৬–১০ রাত:
('guest-17', 'room-A', '[2027-06-01,2027-06-06)', true)
('guest-17', 'room-B', '[2027-06-06,2027-06-11)', true)
```

- `booking_room` থাকবে রুমের দাবি হিসেবে (মালিক, occupancy, তারিখ)। মালিক আর লিঙ্গের constraint এখানেই চলবে।
- এক রাতে একটা রুমে কয়টা সিট ভরা = সেই রাতের `guest_stay` row-এর সংখ্যা (যেখানে `occupies_seat = true`)। সেকশন ১০-এর লক করা যাচাই এটাই গোনে। সিটের সংখ্যা আর আলাদা করে রাখার দরকার নেই।
- রুম বদল বা আগে চেক-আউট মানে শুধু একটা row ছোট করা আর দরকার হলে আরেকটা যোগ করা।
- এজেন্সির তথ্য ছাড়া বুকিং-এও প্রতিটা slot-এর `booking_guest` আর `guest_stay` শুরু থেকেই থাকবে, শুধু `pilgrim_id` খালি।

> এটা Phase 2a-তেই করতে হবে। Phase 2e-তে (বদল ও বাতিল) গিয়ে করলে তখন চালু থাকা বুকিং-এর ওপর schema migration লাগবে।

২ বছরের কম বয়সী শিশু

কোলের বাচ্চার বেড নেই, সিট বা দামে গোনা হবে না, কিন্তু তার পাসপোর্টের তথ্য আর চেক-ইন লাগবে। তাই সে একটা `booking_guest` (`age_group = INFANT`), যার `guest_stay`-তে `occupies_seat = false`। ক্যাপাসিটি, কোটা আর প্রতি রাতের দামের হিসাব থেকে এই row বাদ থাকবে।

এক রুমে এক মালিক

মালিকের জন্য দুইটা আসল foreign key কলাম, ঠিক একটা ভরা থাকবে:

```
owner_agency_id  uuid,   -- এজেন্সির গ্রুপ বুকিং (একই এজেন্সির বুকিংগুলো রুম ভাগ করতে পারে)
owner_booking_id uuid,   -- ফ্যামিলি, আর সাধারণ ইউজারের বুকিং (রুম শুধু এই বুকিং-এর)
CHECK (num_nonnulls(owner_agency_id, owner_booking_id) = 1),
CHECK (owner_booking_id IS NULL OR owner_booking_id = booking_id),  -- অন্য বুকিং-কে দেখাতে
পারবে না
FOREIGN KEY (organization_id, owner_agency_id)  REFERENCES agency  (organization_id,
id),
FOREIGN KEY (organization_id, owner_booking_id) REFERENCES booking (organization_id,
id),

-- constraint-এর জন্য একটা generated কলাম
owner_key text GENERATED ALWAYS AS (
  COALESCE('a:' || owner_agency_id::text, 'b:' || owner_booking_id::text)
) STORED
```

তারপর `btree_gist` দিয়ে (ক্লায়েন্টের সিস্টেমে আগে থেকেই আছে) দুইটা নিয়ম সরাসরি ডাটাবেসে আটকানো, কোডে বাগ থাকলেও ভাঙবে না:

```
-- একই রুমে, ওভারল্যাপ করা তারিখে, দুই মালিক থাকতে পারবে না
ALTER TABLE booking_room ADD CONSTRAINT one_owner_per_room
  EXCLUDE USING gist (room_id WITH =, stay WITH &&, owner_key WITH <>)
  WHERE (status IN ('HELD', 'REQUESTED', 'CONFIRMED', 'IN_HOUSE'));

-- একই রুমে, ওভারল্যাপ করা তারিখে, দুই ধরনের occupancy (পুরুষ/মহিলা/ফ্যামিলি) নয়
ALTER TABLE booking_room ADD CONSTRAINT one_occupancy_per_room
  EXCLUDE USING gist (room_id WITH =, stay WITH &&, occupancy WITH <>)
  WHERE (status IN ('HELD', 'REQUESTED', 'CONFIRMED', 'IN_HOUSE'));
```

(এই constraint-গুলো আসল PostgreSQL 16-এ চালিয়ে যাচাই করা: অন্য এজেন্সি, অন্য লিঙ্গ বা অন্য ফ্যামিলি একই রুমে ঢুকতে পারে না; একই এজেন্সি পারে; বাতিল row রুম আটকায় না।)

প্রতিটা বুকিং-এর নিজের `booking_room`

"নিজের রুম আগে" নিয়মে বুকিং B যদি বুকিং A-এর রুমের খালি সিটে বসে, আর B-এর `guest_stay` A-এর `booking_room` row-কে দেখায়, তাহলে A বাতিল হলে সেই দাবি আর constraint-এ গোনা হবে না। B-এর হাজিরা রুমে থাকবে, কিন্তু রুমটা আর সুরক্ষিত থাকবে না। তাই:

```
-- প্রতিটা বুকিং নিজের booking_room row পাবে (একই owner_key চলবে)
-- guest_stay-এর হাজি আর রুম-দাবি দুইটাই একই বুকিং-এর হতে হবে:
FOREIGN KEY (organization_id, booking_id, booking_guest_id)
  REFERENCES booking_guest (organization_id, booking_id, id),
FOREIGN KEY (organization_id, booking_id, booking_room_id)
  REFERENCES booking_room  (organization_id, booking_id, id)
```

`booking_room.stay` সবসময় তার `guest_stay` row-গুলোর পুরো সময় জুড়ে থাকবে। কোনো রুমের সবাই আগে চেক-আউট করলে দাবিটাও ছোট করা হবে, নইলে অন্য মালিকের জন্য রুমটা অকারণে আটকে থাকবে।

এক রাতে রুমে কয়টা সিট ভরা: একটাই নিয়ম

মেয়াদোত্তীর্ণ হোল্ড, বাতিল বুকিং বা বাদ দেওয়া হাজির `guest_stay` row-ও থেকে যায়। তাই গোনার সময় স্ট্যাটাস দেখতে হয়। এই নিয়ম থাকবে এক জায়গায়, একটা view-তে, যেটা অ্যালোকেশন আর ক্যালেন্ডার দুইটাই ব্যবহার করবে:

```
CREATE VIEW room_night_occupancy AS
SELECT br.organization_id, br.room_id, n.night::date AS night, count(*) AS seats_used
FROM guest_stay gs
JOIN booking_room  br ON br.organization_id = gs.organization_id AND br.id =
gs.booking_room_id
JOIN booking       b  ON b.organization_id  = br.organization_id AND b.id  =
br.booking_id
JOIN booking_guest g  ON g.organization_id  = gs.organization_id AND g.id  =
gs.booking_guest_id
CROSS JOIN LATERAL generate_series(lower(gs.nights), upper(gs.nights) - 1, interval '1
day') AS n(night)
WHERE gs.occupies_seat                                          -- কোলের বাচ্চা বাদ
  AND br.status IN ('HELD', 'REQUESTED', 'CONFIRMED', 'IN_HOUSE')
  AND g.status <> 'REMOVED'
  AND NOT (br.status IN ('HELD', 'REQUESTED') AND b.hold_expires_at <= now())  --
মেয়াদোত্তীর্ণ হোল্ড বাদ
GROUP BY 1, 2, 3;
```

constraint-এর মতো এখানে `now()` ব্যবহার করা যায়, তাই মেয়াদোত্তীর্ণ হোল্ড sweep-এর আগেই গোনা থেকে বাদ পড়ে। আগে চেক-আউট বা no-show-তে সিট ছাড়া মানে `guest_stay` ছোট করা, view নিজে থেকেই তা ধরে। (view আর composite FK আসল PostgreSQL 16-এ চালিয়ে যাচাই করা।)

যা constraint দিয়ে ধরা যায় না, সেগুলো সেকশন ১০-এর লক করা transaction-এ এই view দিয়ে যাচাই হবে: রুমের মোট সিট (ক্যাপাসিটি + এক্সট্রা বেড), কোটা, আর "নিজের রুম আগে" শুধু একই তারিখে।

হাজির তথ্য: স্টাফের master আর বুকারের নিজের কপি

সংস্করণ ৬-এ `pilgrim` পুরো organization-এ একটাই ছিল, আর `passport_lookup` unique। তাতে এজেন্সি B যদি এমন পাসপোর্ট দেয় যেটা এজেন্সি A আগেই দিয়েছে, B-এর slot A-এর রেকর্ডের সাথে যুক্ত হয়ে যেত। B নিজের বুকিং দিয়েই A-এর দেওয়া ঠিকানা ইত্যাদি দেখতে পেত, আর B কোনো বানান ঠিক করলে A-এর রেকর্ডও বদলে যেত। এখন:

- **প্রতিটা বুকার নিজের কপিতে লেখে:** বুকার যা দেয় তা থাকে `booking_guest_detail`-এ, ওই বুকিং-এর সাথে, এনক্রিপ্টেড। এজেন্সি শুধু নিজের কপি পড়তে আর বদলাতে পারে।
- **`pilgrim` শুধু স্টাফের master পরিচয়।** কোনো এজেন্সি বা সাধারণ ইউজার এটা কখনো পড়ে না।
- **মেলানো চুপচাপ, ব্যাকএন্ডে:** নতুন কপি এলে `passport_lookup` দিয়ে master খোঁজা হয়। মিললে লিংক হয়, না মিললে নতুন master। অমিল থাকলে (যেমন দুই কপিতে দুই রকম বানান) স্টাফ দেখে ঠিক করবে।
- **"একই পাসপোর্ট দুই বুকিং-এ" সতর্কতা** শুধু স্টাফ দেখবে। এজেন্সি কখনো জানবে না অন্য কোন এজেন্সি একই হাজিকে দিয়েছে।
- ট্রান্সপোর্টসহ ভবিষ্যতের মডিউল master `pilgrim` ব্যবহার করবে।

পাসপোর্ট নম্বর খোঁজা: `passport_lookup`

পাসপোর্ট নম্বর এনক্রিপ্টেড, তাই index বা `WHERE` দিয়ে খোঁজা যায় না। তাই পাশে একটা keyed হ্যাশ:

```
passport_lookup = HMAC(secret, nationality + normalized_passport_no)

UNIQUE (organization_id, passport_lookup)   -- শুধু master pilgrim-এ
INDEX  (organization_id, passport_lookup)   -- booking_guest_detail-এ, unique নয়
```

`secret` থাকবে এনক্রিপশন কী-র মতোই নিরাপদ জায়গায়। হ্যাশ থেকে নম্বর ফেরত বের করা যায় না।

লগে পাসপোর্টের তথ্য নয়

`audit_log` append-only (trigger দিয়ে আটকানো, মোছা যায় না), আর `booking_change` -এ আগে/পরের JSON থাকে। এগুলোতে পাসপোর্ট নম্বর বা হাজির ব্যক্তিগত তথ্য ঢুকলে ১২ মাসের মোছার নিয়মের পরেও সেগুলো থেকে যাবে, এনক্রিপশন ছাড়াই। তাই:

- দুই জায়গায় লেখার আগে PII ফিল্ড লুকানো বা বাদ (যেমন শুধু `pilgrim_id` আর "পাসপোর্ট নম্বর বদলেছে", নম্বর নয়)।
- একটা টেস্ট যেটা নিশ্চিত করে যে কোনো লগ row-তে পাসপোর্ট নম্বর বা জন্মতারিখ নেই।

১৭. পুরো উদাহরণ

উদাহরণ ১: এজেন্সি, ৩০ জন, হাজির তথ্য ছাড়া

"আল-নূর" এক মাস আগে জানে ৩০ জন হাজি আনবে, কিন্তু পাসপোর্ট এখনো হাতে নেই। WhatsApp-এ কোট নিয়েছে: জনপ্রতি রাত ৩৫০ SAR (পাবলিক দাম ৩৮০)। গ্রুপ বুকিং: ১৮ জন পুরুষ, ১২ জন মহিলা। রুম-মিক্স নিয়ম (১১–২০ জন): ৫ জনের রুম সর্বোচ্চ ২টা।

গ্রুপ	সেরা কম্বিনেশন	খালি সিট	রুম
পুরুষ ১৮	২টা ৫ জনের + ২টা ৪ জনের = ১৮	০	৪
মহিলা ১২	২টা ৫ জনের + ১টা ২ জনের = ১২ বিকল্প: ৩টা ৪ জনের = ১২	০	৩

বুকিং CONFIRMED, ৩০টা slot সব PENDING_DETAILS। পরে এজেন্সি ১০ জনের পাসপোর্টের ছবি দেয়। বাকি ২০ জনের তথ্য হোটেলে পৌঁছালে রিসেপশন ছবি তুলে দেয়।

উদাহরণ ২: নিজের রুম আগে, শুধু একই তারিখে

আল-নূরের ১–১০ তারিখের বুকিং-এ একটা ৪ জনের পুরুষ রুমে ৩ জন আছে, অর্থাৎ ১টা সিট খালি।

নতুন বুকিং	ফলাফল
আল-নূর, ৩ জন পুরুষ, **১–১০**	১ জন যাবে নিজের ওই খালি সিটে, বাকি ২ জনের জন্য একটা ২ জনের রুম।
আল-নূর, ৩ জন পুরুষ, **৩–১২**	তারিখ আলাদা, তাই খালি সিটটা ব্যবহার হবে না। একটা নতুন ৩ জনের রুম।
আল-হারামাইন, ৩ জন পুরুষ, ১–১০	অন্য এজেন্সি, খালি সিটটা পাবে না। একটা নতুন ৩ জনের রুম।

উদাহরণ ৩: সাধারণ ইউজার, ফ্যামিলি

- **স্বামী-স্ত্রী আর দুই সন্তান, ৪ জন:** একটা ৪ জনের রুম, FAMILY। পাবলিক দাম। বুকিং-এর সময় অন্তত বাবার পাসপোর্ট দিতে হবে। বুকিং REQUESTED হবে, ম্যানেজার নিশ্চিত করলে CONFIRMED।
- **বড় পরিবার, ৭ জন:** সিস্টেম দুইটা অপশন দেখাবে: ৫ জনের + ২ জনের রুম, অথবা ৪ জনের + ৩ জনের রুম (দুইটাতেই খালি সিট ০)। পরিবার যেটা বাছবে, দুইটা রুমই শুধু এই বুকিং-এর।

উদাহরণ ৪: সিট কম পড়েছে

হাজি ২০ জন (পুরুষ)। খালি আছে শুধু ৫ জনের ২টা আর ৩ জনের ৩টা রুম, মোট ১৯ সিট। ৩ জনের রুমে ১টা এক্সট্রা বেড চলে।

অপশন	রুম	এক্সট্রা বেড	মোট
একমাত্র অপশন	২টা ৫ জনের + ৩টা ৩ জনের	১টা (৩ জনের একটা রুমে)	২০ ✓

উদাহরণ ৫: এক্সট্রা বেডে একটা রুম কম

হাজি ১৯ জন (পুরুষ)। খালি আছে ৫ জনের ২টা আর ৪ জনের ৫টা রুম। নিয়ম: ৫ জনের রুম সর্বোচ্চ ২টা। ৪ জনের রুমে ১টা এক্সট্রা বেড চলে। দাম রুম হিসেবে হলে (উদাহরণ): ৫ জনের রুম ৫০০, ৪ জনের রুম ৪২০, এক্সট্রা বেড ৮০ SAR প্রতি রাতে।

অপশন	রুম	এক্সট্রা বেড	খালি সিট	রুম	খরচ/রাত
A	৫টা ৪ জনের	০	১	৫	২,১০০ SAR
B	২টা ৫ জনের + ২টা ৪ জনের	১	০	৪	১,৯২০ SAR

অপশন B-তে একটা রুম কম লাগছে, তাই এক্সট্রা বেডের অপশনটাও তালিকায় আসবে। বুকার যেটা বাছবে, সেটাই বুক হবে।

১৮. Monorepo বনাম আলাদা রিপো: বিস্তারিত তুলনা

> **Monorepo মানে সব কোড এক প্রজেক্টে মিশিয়ে ফেলা না।** Monorepo-তেও ব্যাকএন্ড (`apps/api`) আর প্রতিটা ফ্রন্টএন্ড আলাদা প্রজেক্টই থাকবে। প্রতিটার নিজের `package.json` থাকবে, build আর deploy আলাদা হবে, চাইলে আলাদা সার্ভারেও চলবে। পার্থক্য শুধু একটাই: **কোডগুলো একটা Git রিপোর ভেতরে থাকবে, নাকি আলাদা আলাদা রিপোতে।**

একটা উদাহরণ দিয়ে দেখি

ধরেন বুকিং-এ নতুন একটা ফিল্ড যোগ করবেন, যেমন `roomType` (Male / Female / Family)। এটা ব্যাকএন্ড, agency-portal আর ops-console তিন জায়গাতেই লাগবে।

আলাদা রিপো হলে

1. ব্যাকএন্ড রিপোতে DTO বদলাবেন, PR দেবেন, deploy করবেন।
2. agency-portal রিপোতে গিয়ে TypeScript টাইপ হাতে আপডেট করবেন (অথবা shared npm package-এর ভার্সন বাড়াবেন), তারপর আরেকটা PR।
3. ops-console রিপোতেও একই কাজ, আরেকটা PR।
4. কোনো একটা রিপোতে আপডেট করতে ভুলে গেলে TypeScript কিছু বলবে না। **বাগটা ধরা পড়বে প্রোডাকশনে, রানটাইমে।**

Monorepo হলে

1. `packages/contracts` -এ টাইপ একবার বদলাবেন।
2. সাথে সাথে যেসব ফ্রন্টএন্ডে এটা হ্যান্ডেল করা হয়নি, সেখানে **TypeScript error দেখাবে, build-ই হবে না।**
3. সব পরিবর্তন একটা PR-এ যাবে, একসাথে রিভিউ হবে, আর ব্যাকএন্ড ও ফ্রন্টএন্ডের মধ্যে অমিল থাকবে না।

Monorepo-র সবচেয়ে বড় সুবিধা এটাই।

Monorepo-র সুবিধা

- **টাইপ আর ভ্যালিডেশন শেয়ার করা যায়:** একই Zod schema দিয়ে ব্যাকএন্ডে ভ্যালিডেশন হবে, ফ্রন্টএন্ডের ফর্মেও হবে। একবার লিখলেই হয়।
- **একটা ফিচার, একটা PR:** একটা কাজ যদি ৩টা অ্যাপ জুড়ে থাকে, তবু একটা PR-এই হয়ে যায়।
- **শেয়ার করা কোড একবার লিখলেই হয়:** UI কম্পোনেন্ট, তারিখের হিসাব (হিজরি/ইংরেজি), পাসপোর্ট ভ্যালিডেশন, ESLint/tsconfig সব এক জায়গায় থাকে।
- **লাইব্রেরির ভার্সন এক থাকে:** সব অ্যাপে Nuxt, Vue আর অন্য লাইব্রেরির ভার্সন একই থাকে, তাই অ্যাপভেদে আলাদা অদ্ভুত বাগ কম হয়।
- **রিফ্যাক্টর নিরাপদ:** কোনো ফাংশনের নাম বদলালে সব জায়গায় একসাথে বদলে যায়।
- **নতুন ডেভেলপার দ্রুত শুরু করতে পারে:** একবার `git clone` আর `pnpm install` করলেই পুরো সিস্টেম লোকালি চলে।

- **দ্রুত build:** Turborepo শুধু বদলানো অংশটাই build করে, বাকিটা cache থেকে নেয়।

Monorepo-র অসুবিধা (ভবিষ্যতে যেগুলো সামনে আসতে পারে)

- **অ্যাক্সেস কন্ট্রোল:** GitHub-এ পারমিশন দেওয়া যায় পুরো রিপোর জন্য। কোনো ফ্রিল্যান্সারকে শুধু public-site-এর কাজ দিলেও সে পুরো কোড দেখতে পাবে। CODEOWNERS দিয়ে কে কোন অংশ রিভিউ করবে সেটা ঠিক করা যায়, কিন্তু কোড দেখা আটকানো যায় না।
- **Tooling শিখতে হবে:** pnpm workspace, Turborepo, আর Docker build-এ `turbo prune` শুরুতে একটু ঝামেলার লাগবে।
- **CI ঠিকমতো সেট না করলে সব সময় সব build হবে:** "শুধু যেটা বদলেছে সেটাই build/deploy" এই কনফিগারেশন করে নিতে হয়।
- **ভুলভাল import হওয়ার ঝুঁকি:** খুব সহজে এক অ্যাপ থেকে আরেক অ্যাপের কোড import করা যায়, ফলে কোড জট পাকিয়ে যেতে পারে। এটা আটকাতে lint rule দিয়ে নিয়ম বেঁধে দিতে হবে।
- **বড় আপগ্রেড একসাথে করতে হয়:** যেমন Nuxt-এর বড় ভার্সন আপগ্রেড করলে সব অ্যাপ একসাথে আপগ্রেড করতে হবে। এটা ভালোও, কারণ কোনো অ্যাপ পুরোনো ভার্সনে পড়ে থাকে না।
- **রিপো বড় হতে থাকবে:** ১০–১৫টা অ্যাপ হয়ে গেলে git history আর CI একটু ভারী হবে। তবে আপনার স্কেলে এটা অনেক দূরের ব্যাপার।

আলাদা রিপোর সুবিধা

- **পুরো স্বাধীনতা:** প্রতিটা অ্যাপের নিজস্ব রিলিজ সাইকেল আর নিজস্ব CI থাকে, যেটা সহজ আর ছোট।
- **অ্যাক্সেস কন্ট্রোল সহজ:** কোনো একটা অ্যাপ আউটসোর্স করলে শুধু ওই রিপো দিলেই হয়।
- **আলাদা টেক স্ট্যাক ব্যবহার করা যায়:** যেমন রিসেপশনের জন্য পরে Flutter দিয়ে মোবাইল অ্যাপ বানাতে চাইলে সেটা নিজের রিপোতে থাকতে পারে।
- **সীমানা জোর করে আলাদা থাকে:** এক অ্যাপ আরেক অ্যাপের কোড ভুল করে ব্যবহার করতে পারে না।

আলাদা রিপোর অসুবিধা

- **টাইপ বা API কন্ট্রাক্টে অমিল:** ব্যাকএন্ড বদলালো কিন্তু ফ্রন্টএন্ড জানতে পারলো না। এই বাগগুলো সবচেয়ে বেশি কষ্ট দেয়।
- **একটা ফিচারের জন্য ৩–৪টা PR:** কোনটা আগে deploy হবে সেটা ঠিক রাখতে হয়, ভুল ক্রমে deploy হলে কিছু সময়ের জন্য সিস্টেম ভেঙে থাকে।
- **কপি-পেস্ট:** একই কম্পোনেন্ট আর utility ৩ জায়গায় থাকে, আর সময়ের সাথে তিনটা তিন রকম হয়ে যায়।
- **Shared package-এর ভার্সন সামলানো কঠিন:** `@zatiq/types` v1.2 পাবলিশ করা, তারপর প্রতিটা রিপোতে আপডেট করা, কোনো অ্যাপ পুরোনো ভার্সনে রয়ে যাওয়া।

পাশাপাশি তুলনা

বিষয়	Monorepo	আলাদা রিপো
টাইপ শেয়ার	স্বয়ংক্রিয়, compile time-এ ভুল ধরা পড়ে	npm package বা কপি করতে হয়, ভুল ধরা পড়ে রানটাইমে
একটা ফিচার একাধিক অ্যাপে	১টা PR	একাধিক PR + deploy-এর ক্রম মেলাতে হয়
শুরুতে সেটআপ	একটু বেশি	সহজ
অ্যাক্সেস কন্ট্রোল	পুরো রিপো একসাথে	প্রতিটা রিপো আলাদা
আলাদা টেক স্ট্যাক	সম্ভব, তবে সব TS হলে সবচেয়ে ভালো	সহজ
ছোট টিম (১–৮ জন)	✓ সবচেয়ে উপযুক্ত	বাড়তি ঝামেলা
বড় আলাদা আলাদা টিম	কনফিগারেশন ঠিক রাখতে হয়	✓ স্বাভাবিক
পারফরম্যান্স (ইউজারের দিক থেকে)	**কোনো পার্থক্য নেই**	**কোনো পার্থক্য নেই**

> শেষ লাইনটা গুরুত্বপূর্ণ। **ওয়েবসাইট কতটা দ্রুত চলবে, তার সাথে monorepo বা আলাদা রিপোর কোনো সম্পর্ক নেই।** ইউজারের কাছে একই build-করা অ্যাপ যায়। পারফরম্যান্স নির্ভর করে DB ডিজাইন, query, caching, CDN আর ফ্রন্টএন্ডের bundle size-এর উপর। রিপোর গঠন শুধু **ডেভেলপমেন্টের গতি আর বাগ কম হওয়ার** উপর প্রভাব ফেলে।

আপনার ক্ষেত্রে কোনটা ভালো

আপনার অবস্থা হলো: টিম ছোট, সব কোড TypeScript (Nuxt + Vue), আর সব অ্যাপ একই হাজির ডেটা আর একই বুকিং নিয়ে কাজ করে। **এই অবস্থায় Monorepo পরিষ্কারভাবে ভালো।**

আলাদা রিপোতে যাওয়া উচিত যখন:

- আলাদা টিম আলাদা অ্যাপের দায়িত্ব নেবে, আর তারা নিজেদের মতো রিলিজ করবে।
- কোনো অংশ বাইরের কোম্পানিকে দিয়ে বানাবেন, আর পুরো কোড দেখাতে চান না।
- কোনো অ্যাপ সম্পূর্ণ আলাদা টেকনোলজিতে যাবে (যেমন native mobile)।
- কোনো সিস্টেম, যেমন transport, আলাদা প্রোডাক্ট হিসেবে অন্য ক্লায়েন্টদের কাছে বিক্রি করবেন।

মাঝামাঝি একটা অপশনও আছে: আলাদা রিপো রেখেও টাইপের অমিল অনেকটা কমানো যায়। API থেকে OpenAPI spec বানিয়ে প্রতিটা ফ্রন্টএন্ডে `openapi-typescript` বা `orval` দিয়ে typed API client generate করবেন। তবে এখানেও spec পাবলিশ করা আর প্রতিটা রিপোতে regenerate করার একটা বাড়তি ধাপ থেকে যায়।

> **আরেকটা ব্যবহারিক সুবিধা:** Monorepo থেকে পরে কোনো অ্যাপ আলাদা রিপোতে সরানো সহজ, ফোল্ডারটা history-সহ বের করে নিলেই হয় (`git subtree split`)। কিন্তু আলাদা আলাদা রিপো পরে এক জায়গায় আনা তুলনামূলক ঝামেলার। তাই Monorepo দিয়ে শুরু করলে ভবিষ্যতে দুই দিকেই যাওয়ার পথ খোলা থাকে।

১৯. Hajj & Umrah Hotel Operations System: বর্তমান বিজনেস লজিক

প্রজেক্টের সারসংক্ষেপ

এটি সৌদি আরবের মক্কা ও মদিনায় পরিচালিত **হজ ও উমরাহ হোটেলগুলোর জন্য একটি মাল্টি-হোটেল অপারেশনাল ম্যানেজমেন্ট প্ল্যাটফর্ম** (B2B/B2C)। সাধারণ হোটেল সফটওয়্যার (PMS) থেকে হজ-উমরাহ হোটেলের কাজের ধরন অনেক আলাদা। এই সিস্টেম মূলত তিনটা চ্যালেঞ্জ সমাধানের জন্য বানানো:

1. **তারিখ বা সিজনভিত্তিক ক্যাপাসিটি (Date-effective Capacity):** সাধারণ হোটেলে একটা রুমের বেড সারা বছর একই থাকে। হজ বা রমজানের পিক সিজনে একই রুমে বাড়তি খাট দিয়ে ধারণক্ষমতা বাড়ানো হয়, অফ-সিজনে আবার কমে যায়।
2. **রুমের আগে হাজির সংখ্যা (Pilgrim Count First):** এজেন্সি "৪টা রুম" খোঁজে না, বলে "আমাদের ৫০ জন হাজি আছেন"। সিস্টেমকে এই সংখ্যার ভিত্তিতে সবচেয়ে কার্যকর রুম-কম্বিনেশন বানাতে হয়।
3. **হিস্ট্রি ও ডেটার বিশ্বস্ততা (Single Source of Truth):** অতীত ও ভবিষ্যতের ইনভেন্টরি, ক্যাপাসিটি পরিবর্তন আর ফিনান্সিয়াল ডেটার কোনো হিস্ট্রি মোছা হয় না। সবকিছু ভার্সন আর ডেট-রেঞ্জ আকারে থাকে।

বর্তমান অবস্থা: Phase 1 সম্পন্ন

অবস্থা	বিবরণ
তৈরি হয়েছে	**Phase 1 (Hotel Inventory Foundation)-এর ব্যাকএন্ড।** মাল্টি-টেন্যান্ট সিকিউরিটি, হোটেল ও ফ্লোর ম্যানেজমেন্ট, রুম টাইপ, ভার্সনসহ ক্যাপাসিটি, সিজনাল ওভাররাইড (হজ/রমজান), অপারেশনাল ব্লক (রক্ষণাবেক্ষণ/মেরামত), রুম ক্যালেন্ডার ইঞ্জিন, অডিট লগ, হোটেল ডকুমেন্ট আর একটা ডেমো অর্গানাইজেশন।
এখনো হয়নি	ফ্রন্টএন্ড UI এখনো শুরুর (Phase 0 scaffolding) অবস্থায়, তবে UI/UX-এর পূর্ণ দিকনির্দেশনা লেখা আছে। বুকিং/রিজার্ভেশন ইঞ্জিন হবে **Phase 2**-তে।

টেকনোলজি স্ট্যাক

মাইক্রোসার্ভিসের জটিলতা এড়িয়ে **Modular Monolith** আর্কিটেকচার অনুসরণ করা হয়েছে।

উপাদান	প্রযুক্তি	বিবরণ
ফ্রেমওয়ার্ক	Nuxt 4 / Nitro (v4.5.2)	ফুল-স্ট্যাক TypeScript ফ্রেমওয়ার্ক
ভাষা ও রানটাইম	TypeScript + Node.js (≥22.11)	পুরো কোডবেস টাইপ-সেফ
ডাটাবেস	PostgreSQL 16	একক সোর্স অব ট্রুথ। ডেট-রেঞ্জ ওভারল্যাপ আটকাতে `btree_gist` এক্সটেনশন
ORM	Drizzle ORM (v0.45)	SQL-ফার্স্ট পারফরম্যান্স আর নেটিভ Postgres কনস্ট্রেইন্ট নিয়ন্ত্রণ
অথেনটিকেশন	nuxt-auth-utils + Argon2id	সিলড কুকি সেশন, আলাদা সেশন টেবিল নেই
ভ্যালিডেশন	Zod (v3.25)	সার্ভার ও ক্লায়েন্টে শেয়ার করা ভ্যালিডেশন স্কিমা
টেস্টিং	Vitest	ইউনিট, ইন্টিগ্রেশন আর আসল সার্ভারে HTTP টেস্ট
প্যাকেজ ম্যানেজার	pnpm (v9.4.0)	দ্রুত ও কঠোর ডিপেন্ডেন্সি ম্যানেজমেন্ট

ডাটাবেস ও মূল ডোমেইন মডেল

ক. টেন্যান্সি ও নিরাপত্তা

- `organization`: প্রতিটা ক্লায়েন্ট/সংস্থা একেকটা স্বাধীন টেন্যান্ট।
- `app_user`: সংস্থার ইউজাররা।
- `role`, `permission`, `user_role`: কঠোর RBAC (Role-Based Access Control)।
- **ডেটা লিকেজ রোধ:** প্রতিটা টেবিলে কম্পোজিট ফরেন কি (`organization_id, id`)। কেউ এমন হোটেলের আইডি দিয়ে রিকোয়েস্ট করলে যার অ্যাক্সেস তার নেই, সিস্টেম `403 Forbidden` না দিয়ে `404 Not Found` দেয়। ফলে হোটেলটা আছে কিনা সেটাও অনুমান করা যায় না।

খ. হোটেল ও ফ্লোর

- `hotel`: নাম, কোড, শহর (Makkah/Madinah), টাইমজোন (`Asia/Riyadh`), কারেন্সি (`SAR`), চেক-ইন/আউট সময়, লাইসেন্স রেফারেন্স, মালিকানার ধরন (`OWNED`, `LEASED`, `CONTRACTED`) আর স্ট্যাটাস (`ACTIVE`, `INACTIVE`)।
- `hotel_setting`: প্রতিটা হোটেলের আলাদা কনফিগারেশন, যেমন রক্ষণাবেক্ষণের সময় রুম বিক্রি বন্ধ থাকবে কিনা (`inventory.maintenanceBlocksSales`)।
- `floor`: লেভেল (−৫ থেকে ২০০, বেসমেন্টসহ) আর লেবেল।

গ. ইনভেন্টরি ও ক্যাপাসিটি ইঞ্জিন (সিস্টেমের সবচেয়ে জটিল অংশ)

1. `room_type`: সংস্থার রুম ক্যাটাগরি (Quad, Triple, Double ইত্যাদি), সাথে ডিফল্ট বেড আর হাজি ক্যাপাসিটি।
2. `room`: ফিজিক্যাল রুম। এখানে কোনো স্থির `status` কলাম (Available, Occupied) নেই। রুমের অবস্থা তারিখ অনুযায়ী হিসাব করে বের করা হয়। রুম নম্বর একবার তৈরি হলে আর বদলানো যায় না।

3. `room_base_config` (ভার্সনসহ ক্যাপাসিটি): স্বাভাবিক অবস্থায় রুমে কয়টা বেড আর কতজন হাজি থাকতে পারে। `valid_from` থেকে `valid_to` রেঞ্জ দিয়ে হিস্ট্রি রাখা হয়। যেমন রেনোভেশনে ৩ বেড থেকে ৪ বেড হলে আগের হিস্ট্রি থেকে যায়। Postgres-এর `EXCLUDE USING gist` কনস্ট্রেইন্ট নিশ্চিত করে যে একই রুমের দুইটা ভার্সনের তারিখ কখনো ওভারল্যাপ করবে না।
4. `capacity_period` ও `room_capacity_override`: রমজান, হজ বা বিশেষ পিক সিজনে নির্দিষ্ট সময়ের জন্য ধারণক্ষমতা বদলানোর ব্যবস্থা। যেমন সাধারণ সময়ে Quad রুমে ৪ জন, কিন্তু হজে ৬ জন।
5. `room_operational_block`: রুম ব্যবহারের অনুপযোগী থাকলে (`OUT_OF_SERVICE`, `MAINTENANCE`, `OPERATIONAL_BLOCK`)। ব্লক কখনো সরাসরি মোছা হয় না, সফট-ক্যানসেল বা `ended_early` করা হয়।

ঘ. ক্যালেন্ডার ও দৈনিক সামারি অ্যালগরিদম

প্রতিদিনের জন্য আলাদা রো স্টোর না করে সিস্টেম **Run-Length Encoding (RLE)** আর **Epoch Day** পদ্ধতিতে খুব দ্রুত একটা হোটেলের ৪০০ দিনের ক্যালেন্ডার আর দৈনিক সামারি বানায়: কয়টা রুম খালি, কয়টা ব্লকে, মোট বিক্রিযোগ্য ক্যাপাসিটি কত।

ঙ. অডিট লগ ও ডকুমেন্ট

- `audit_log`: সব পরিবর্তন ট্র্যাক হয়। Postgres ট্রিগার নিশ্চিত করে যে অডিট ডেটা কেউ এডিট বা মুছতে পারবে না (append-only)।
- `document_asset` ও `hotel_document`: হোটেলের লাইসেন্স, চুক্তিপত্র, ইন্স্যুরেন্স ইত্যাদি আপলোড ও সংরক্ষণ। মেটাডেটা থাকে ডাটাবেসে, ফাইল থাকে স্টোরেজে (Local/S3), সাথে SHA-256 চেকসাম।

বিল্ট-ইন ডেমো অর্গানাইজেশন

- **৫টা হোটেল:** মক্কায় ৩টা (`MKK-GRAND`, `MKK-AJYAD`, `MKK-AZIZ`), মদিনায় ২টা (`MED-CENT`, `MED-QUBA`)।
- **৩৬০টা রুম**, ৪টা রুম টাইপ, **২৪টা ক্যাপাসিটি পিরিয়ড** (রমজান, হজ ইত্যাদি) আর **৮৩০টা সিজনাল রুম ওভাররাইড**।
- **৯টা ইউজার রোল/পারসোনা:** `admin` (সব হোটেলের অ্যাক্সেস), `manager.grand` (মক্কার গ্র্যান্ড হোটেল), `manager.madinah` (মদিনার ২টা হোটেল), `reception.grand` ও `reception.ajyad` (শুধু ইনভেন্টরি দেখতে পারে), `accountant`, `hr` ইত্যাদি।
- সব আইডি UUID v5 দিয়ে নির্দিষ্ট নিয়মে তৈরি হয়। তাই ডাটাবেস যতবার রিসেট করা হোক, আইডি বা বুকমার্ক বদলায় না।

ডেভেলপার কমান্ড

কমান্ড	কাজ
`docker compose up -d`	লোকাল PostgreSQL 16 কন্টেইনার চালু করে (`localhost:5433`)
`pnpm db:migrate`	ডাটাবেস মাইগ্রেশন চালায়
`pnpm db:seed`	ডেমো অর্গানাইজেশন তৈরি বা রিসেট করে
`pnpm dev`	লোকাল সার্ভার চালু করে (`http://localhost:3000`)
`pnpm test` / `pnpm test:unit`	ইউনিট টেস্ট (ডাটাবেস ছাড়া, দ্রুত)
`pnpm test:integration`	টেস্ট ডাটাবেসে আসল Postgres query ও কনস্ট্রেইন্ট টেস্ট
`pnpm test:http`	পুরো অ্যাপ build করে আসল HTTP রিকোয়েস্ট দিয়ে end-to-end টেস্ট
`pnpm verify`	CI-এর সব ধাপ (Lint, Typecheck, DB Check, Unit, Integration, HTTP) ক্রমানুসারে চালায়

পরবর্তী পরিকল্পনা (রোডম্যাপ)

ধাপ	কাজ
Phase 1 UI	ড্যাশবোর্ড, ক্যালেন্ডার রিবন ভিউ, ড্রয়ার, আর পূর্ণ আরবি/ইংরেজি ইন্টারফেস
Phase 2	বুকিং ও অ্যালোকেশন: হাজির সংখ্যা দিলে স্বয়ংক্রিয়ভাবে সেরা রুম বরাদ্দ আর রিজার্ভেশন নিশ্চিত করার ইঞ্জিন
Phase 3	ফিনান্স ও ইনভয়েসিং: রেট প্ল্যান, সৌদি ১৫% VAT, ইনভয়েস, পেমেন্ট ট্র্যাকিং
Phase 5	HR ও সৌদি কমপ্লায়েন্স: কর্মীদের ইকামা (Iqama), শ্রম আইন, পেরোল

২০. নতুন নিয়মগুলো বর্তমান সিস্টেমে কীভাবে বসবে

বিষয়	বর্তমান সিস্টেম	যা করতে হবে
আর্কিটেকচার	Nuxt 4 / Nitro, Modular Monolith, PostgreSQL, Drizzle, Zod	সবই থাকবে। শুধু monorepo-তে সরানো (Phase 2-0)।
ফ্রন্টএন্ড ও UI	Phase 0 scaffolding; অনুমোদিত UI ডিরেকশন Nuxt UI v4	তিনটা Nuxt অ্যাপ: public-site (SSR), agency-portal আর ops-console (SPA)।
টেন্যান্সি	`organization_id` + কম্পোজিট FK, tenant-isolation টেস্ট	নতুন সব টেবিলে একই। `AgencyScope` আর `CustomerScope` যোগ, টেস্টে কভার।
ইনভেন্টরি সার্ভিস	রুম/ফ্লোর বন্ধ, ব্লক, ক্যাপাসিটি বদল	বুকিং-এর সাথে সংঘর্ষ চেক। ফ্লোর বন্ধ, রুম আবার চালু আর ক্যাপাসিটি পিরিয়ড মোছায় লক যোগ (সেকশন ১০)।
রাতভিত্তিক ক্যাপাসিটি	ক্যালেন্ডার ইঞ্জিন প্রতি রাতের ক্যাপাসিটি আর ব্লক হিসাব করে	সেকশন ৯.১-এর "ব্যবহারযোগ্য সিট" এখান থেকে।
ডেট-রেঞ্জ constraint	`btree_gist` আর `EXCLUDE` আগে থেকেই ব্যবহার হচ্ছে	`owner_key` আর `occupancy` -এর constraint একই পদ্ধতিতে।
ব্যাকগ্রাউন্ড জব	আর্কিটেকচার ডকে pg-boss পরিকল্পিত	হোল্ডের মেয়াদ ও sweep, রিমাইন্ডার, পাসপোর্ট পড়া, পুরোনো পাসপোর্টের ছবি মোছা।
ক্যালেন্ডার	ক্যালেন্ডার ইঞ্জিন আর Phase 1 UI-তে পরিকল্পিত রিবন ভিউ	অকুপেন্সি ক্যালেন্ডার এর ওপরে, বুকিং আর হাজির তথ্য যোগ করে।
সিজনাল ক্যাপাসিটি	`room_capacity_override`	এক্সট্রা বেড এর থেকে আলাদা (সেকশন ১১)।
ইউজার ও অ্যাক্সেস	শুধু ভেতরের স্টাফ	এজেন্সি (অনুমোদনসহ) আর সাধারণ ইউজার। স্টাফের নতুন অনুমতি: "এজেন্সির পক্ষে বুকিং", "কোটের দাম বসানো"। "অ্যাক্সেস না থাকলে 404" সবার ক্ষেত্রে।
দাম	রেট প্ল্যান Phase 3-এ	**বুকিং দাম ছাড়া চলে না।** পাবলিক দাম, এজেন্সির দাম, কোট আর প্রতি রাতের দাম Phase 2a-তে। ইনভয়েস, VAT, পেমেন্ট Phase 3-তে।
ডকুমেন্ট	`document_asset`, SHA-256, Local/S3	পাসপোর্টের ছবি এখানে, তবে এনক্রিপশন আর কড়া অ্যাক্সেসসহ।
অডিট	append-only `audit_log` (শুধু পরিবর্তন)	সাথে `pii_access_log` (কে কী দেখেছে) আর `booking_change`। দুই লগেই পাসপোর্টের তথ্য লুকানো।
ট্রান্সপোর্ট	রোডম্যাপে নেই	পরে, ops-console-এর একটা অংশ হিসেবে। একই `pilgrim` টেবিল।

২১. বাইরের ইউজার আসার আগে যা লাগবে

এজেন্সি পোর্টাল বা পাবলিক সাইট চালু হওয়ার আগে এগুলো শেষ হতেই হবে। এগুলো এখন ঐচ্ছিক উন্নতি নয়, চালু করার পূর্বশর্ত।

কাজ	কেন
লগইন চেষ্টার সীমা (rate limit)	পাসওয়ার্ড অনুমান করে ঢোকা ঠেকাতে
CSRF origin চেক, তিনটা ডোমেইনের allowlist দিয়ে	কুকি-ভিত্তিক সেশনে অন্য সাইট থেকে আসা রিকোয়েস্ট ঠেকাতে
সাধারণ ইউজারের সাইন-আপে ফোন OTP, দরকারে captcha	বট দিয়ে ভুয়া অ্যাকাউন্ট বানিয়ে রুম আটকানো ঠেকাতে
স্টাফের জন্য 2FA	অ্যাডমিন আর ম্যানেজারের অ্যাকাউন্ট সবচেয়ে মূল্যবান
এজেন্সি অনুমোদন	লাইসেন্স যাচাই ছাড়া কেউ এজেন্সির দাম পাবে না
পাসপোর্টের তথ্যের সুরক্ষা	এনক্রিপশন, দেখার লগ, মোছার নীতি (সেকশন ১৩)
একই ডোমেইনে `/api`	sealed cookie কাজ করে, CORS খোলার দরকার পড়ে না
প্রতিটা API route group-এ ইউজারের ধরন যাচাই (স্টাফ / এজেন্সি / সাধারণ ইউজার)	তিনটা ডোমেইনের কুকি একই `NUXT_SESSION_PASSWORD` দিয়ে sealed। ব্রাউজারে আলাদা হলেও API-র কাছে আলাদা নয়, তাই রিকোয়েস্ট যে ডোমেইন থেকেই আসুক, API নিজে যাচাই করবে।
কুকি কখনো parent domain-এ নয়; প্রতিটা proxy শুধু নিজের route prefix পাঠাবে (যেমন এজেন্সির ডোমেইন শুধু `/api/agency/**`)	এক অ্যাপের সেশন দিয়ে অন্য অ্যাপের API ছোঁয়া ঠেকাতে
পাবলিক সার্চে rate limit, আর মোটামুটি ফলাফল (সঠিক সিট সংখ্যা নয়)	বট দিয়ে দাম তুলে নেওয়া, বা বারবার অ্যালোকেশন ইঞ্জিন চালিয়ে সার্ভার ভারী করা ঠেকাতে
রিসিপ্টের QR-এ র‍্যান্ডম টোকেন	বুকিং আইডি বা ব্যক্তিগত তথ্য নয়। চেক-ইনে সার্ভার টোকেন যাচাই করবে।
হোস্টিং রিজিয়নের আইনি যাচাই	বাহরাইন বা UAE-তে রাখাও PDPL-এ দেশের বাইরে ডেটা পাঠানো। সৌদির ভেতরের রিজিয়ন বিবেচনা।

২২. প্রস্তাবিত বানানোর ক্রম

ধাপ	কাজ
Phase 2-0 monorepo	বর্তমান অ্যাপ → `apps/api`, `shared/schemas` → `packages/contracts`, `db` → `packages/db`। CI-এর path আর HTTP টেস্ট হারনেস আপডেট। কোনো ফিচার বদল নয়, সব টেস্ট পাস করতে হবে।
Phase 1 UI (ক্লায়েন্টের)	ops-console-এ, রোডম্যাপ অনুযায়ী 2c-এর আগে।
Phase 2a ভিত্তি ও নিরাপত্তা	টেন্যান্সিসহ নতুন টেবিল, owner/occupancy constraint, প্রতি রাতের দাম আর কোট, কোটা, অ্যাডমিন সেটিংস, ইনভেন্টরি-সংঘর্ষের নিয়ম আর Phase 1-এর লকের ফাঁক বন্ধ, pg-boss, সেকশন ২১-এর নিরাপত্তা কাজ। **সংস্করণ ৬-এ যোগ:** `guest_stay` আর শিশুর slot, `passport_lookup` হ্যাশ, লগে PII লুকানো, এজেন্সির মধ্যে হাজির তথ্য আলাদা, আর শুরুতেই pg-boss-এর transaction spike। **সংস্করণ ৭-এ যোগ:** master `pilgrim` আর বুকারের কপি (`booking_guest_detail`), প্রতিটা বুকিং-এর নিজের `booking_room` আর composite FK, `room_night_occupancy` view, বুকিং ফর্মে শিশুর সংখ্যা।
Phase 2b অ্যালোকেশন ও হোল্ড	অ্যালোকেশন ইঞ্জিন, DP দিয়ে, এক্সট্রা-জায়গা আর "৫ বা বেশি সিট" সীমাসহ (গ্রুপ, ফ্যামিলি, নিজের রুম একই তারিখে, রুম-মিক্স, এক্সট্রা বেড), সেকশন ১০-এর লক করা হোল্ড, মেয়াদোত্তীর্ণ হোল্ডের sweep। concurrency টেস্ট: একসাথে অনেক রিকোয়েস্ট পাঠিয়ে প্রমাণ করা যে কখনো বেশি বুক হয় না। আসল রুমের সব সম্ভাব্য সেট যাচাই করে DP-র property test, পুরো অপশন তালিকা মিলিয়ে।
Phase 2c এজেন্সি পোর্টাল ও কোট	বুকিং স্ক্রিন, WhatsApp কোট, ম্যানেজারের "এজেন্সির পক্ষে বুকিং", রিসিপ্ট ও QR।
Phase 2d পাসপোর্ট, চেক-ইন, ক্যালেন্ডার	পাসপোর্টের ছবি থেকে তথ্য, রিভিউ ফর্ম, প্রতি হাজির চেক-ইন, এক্সট্রা বেডের কাজ, রিমাইন্ডার, অকুপেন্সি ক্যালেন্ডার। OCR আলাদা worker প্রসেসে।
Phase 2e বদল ও বাতিল	হাজি যোগ/বাদ, তারিখ ও রুম বদল, বাতিল, no-show, আগে চেক-আউট (সেকশন ১২)। Phase 2-এর সবচেয়ে কঠিন অংশ, আলাদা সময় রাখা দরকার।
Phase 2f পাবলিক ওয়েবসাইট	সাধারণ ইউজারের বুকিং: REQUESTED স্ট্যাটাস, OTP, খোলা অনুরোধের সীমা।
Phase 3 ফিনান্স (ক্লায়েন্টের)	ইনভয়েস, ১৫% VAT, পেমেন্ট, বাতিলের চার্জ, এক্সট্রা বেডের চার্জ।
ট্রান্সপোর্ট	ক্লায়েন্টের সাথে ঠিক করার পরে, ops-console-এ।

> **এর পরে:** তিন দফা রিভিউয়ের পর ডিজাইন এখন Phase 2a শুরু করার মতো স্থির। ডক আর না বাড়িয়ে Phase 2-0 (monorepo-তে সরানো) আর pg-boss-এর transaction spike দিয়ে কাজ শুরু করা ভালো। বাকি খুঁটিনাটি আসল কোড আর টেস্টেই দ্রুত ধরা পড়বে।

২৩. ক্লায়েন্টের সাথে যা নিশ্চিত করতে হবে

প্রতিটার পাশে "আপাতত যা ধরে নেওয়া" দেওয়া আছে, যাতে উত্তর আসা পর্যন্ত কাজ আটকে না থাকে।

#	প্রশ্ন	আপাতত যা ধরে নেওয়া
বুকার ও দাম		
১	একটাই অপারেটর (এই ক্লায়েন্ট), নাকি পাবলিক সাইট ভবিষ্যতে একাধিক অপারেটরের মার্কেটপ্লেস?	একটাই অপারেটর, একটা organization।
২	এজেন্সি কি ফ্যামিলি বুকিং করতে পারবে, আর সাধারণ ইউজার কি গ্রুপ?	হ্যাঁ, দুজনেই দুই ধরনই বাছতে পারবে।
৩	দাম জনপ্রতি, নাকি রুম হিসেবে?	গ্রুপ: জনপ্রতি প্রতি রাত। ফ্যামিলি: রুম হিসেবে প্রতি রাত। এক্সট্রা বেড আলাদা।
৪	জনপ্রতি দামে গ্রুপের রুমে সিট খালি থাকলে সেই সিটের দাম কে দেবে?	কেউ না, হোটেলের ক্ষতি। তাই অ্যালগরিদম খালি সিট সবচেয়ে কমায়।
৫	শিশু আর কোলের বাচ্চা কি আলাদা বেড পাবে, দাম কত?	২ বছরের কম বয়সীর বেড নেই, সিটে বা দামে গোনা হবে না। বাকিরা পূর্ণ হাজি হিসেবে।
৬	অনুমোদিত সব এজেন্সির জন্য কি একটা ডিফল্ট এজেন্সি দাম থাকবে?	ঐচ্ছিক। WhatsApp কোট সবসময় চলবে।
৭	কোটের মেয়াদ কত? আর কোট রুম আটকায় না, সেটা কি ঠিক আছে?	৪৮ ঘণ্টা, রুম আটকায় না। এজেন্সিকে সেটা স্পষ্ট দেখানো হবে।
৮	সাধারণ ইউজার কি অনলাইনে পেমেন্ট করবে?	প্রথম সংস্করণে না। বুকিং REQUESTED থাকবে, ম্যানেজার নিশ্চিত করলে CONFIRMED।
৯	সাধারণ ইউজারের সীমা: প্রতি বুকিং-এ কয়টা রুম, কয়টা খোলা অনুরোধ, অনুরোধ কতক্ষণ?	৫টা রুম, ১টা খোলা অনুরোধ, ২৪ ঘণ্টা।
রুম বরাদ্দ		
১০	"একই তারিখ" মানে কি চেক-ইন ও চেক-আউট হুবহু এক? নাকি নতুন থাকার সময় আগেরটার ভেতরে থাকলেও চলবে?	হুবহু এক।
১১	রুম-মিক্স নিয়ম পুরো গ্রুপের ওপর বসবে, নাকি পুরুষ আর মহিলার আলাদা সংখ্যার ওপর?	লিঙ্গভিত্তিক আলাদা সংখ্যার ওপর।
১২	গ্রুপে একা একজন মহিলা হলে তাকে পুরো একটা রুম দিতে হবে। তখন ভাড়া কীভাবে?	সবচেয়ে ছোট খালি রুম (১–২ জনের) আগে। ভাড়ার নিয়ম ক্লায়েন্টের।
১৩	কোটা কি প্রতি সিজনে নির্দিষ্ট সংখ্যা (শতাংশ নয়) হলে চলবে? এজেন্সির ফ্যামিলি বুকিংও কি কোটায় গোনা হবে?	হ্যাঁ, নির্দিষ্ট সংখ্যা, প্রতি রাতের হিসাবে। এজেন্সির সব বুকিং গোনা হবে।

১৪	এজেন্সির রুমে খালি সিট পড়ে থাকলে কি কোনো সময় অন্যদের জন্য ছেড়ে দেওয়া হবে?	না। অ্যাডমিন হাতে ছেড়ে দিতে পারবে।
১৫	এক্সট্রা বেড: দাম কি রুম টাইপ বা সিজনভেদে আলাদা? সিজনাল ক্যাপাসিটির রুমে চলবে?	প্রতি হোটেলে একটা দাম; সিজনাল রুমে না।
১৬	গ্রুপ বুকিং-এ ছোট ছেলে কি মায়ের রুমে থাকতে পারবে? কোন বয়স থেকে পুরুষ-মহিলা আলাদার নিয়ম খাটবে?	১০ বছরের কম বয়সী শিশু মা বা বাবার যেকোনো রুমে থাকতে পারবে; ১০ বা তার বেশি হলে নিজের লিঙ্গের রুমে (অ্যাডমিন সেটিং)। বুকিং ফর্মে শিশুর সংখ্যা আর কার সাথে থাকবে (ডিফল্ট মায়ের সাথে) আলাদা নেওয়া হবে।
১৭	৪০ জনের বেশি গ্রুপের রুম-মিক্স নিয়ম কী?	একই অনুপাতে: প্রতি ১০ জনে ৫ জনের রুম আরও ১টা।
১৮	রুম-মিক্সের সীমা কি "৫ বা তার বেশি সিটের রুম" (হজে ৬ সিট হয়ে যাওয়া রুমসহ), নাকি নির্দিষ্ট রুম টাইপের ওপর?	৫ বা তার বেশি সিটের সব রুম একসাথে।
১৯	যেসব রাত কোনো সিজনে পড়ে না, সেখানে কোটা কী?	ডিফল্ট কোটার row খাটবে; সেটাও না থাকলে কোনো সীমা নেই।
বদল ও বাতিল		
২০	বাতিল নীতি: কত দিন আগে বাতিল করলে কত চার্জ?	ক্লায়েন্ট ঠিক করবে। আপাতত শুধু ম্যানেজার বাতিল করতে পারবে, চার্জ ছাড়া।
২১	এজেন্সি কোন বদল নিজে করতে পারবে?	হাজি যোগ নিজে। বাদ, তারিখ বা রুম বদল আর বাতিল ম্যানেজারের অনুমোদনে।
২২	No-show বা আগে চেক-আউট হলে টাকা ফেরত হবে? সিট ছেড়ে দেওয়া হবে?	টাকা ফেরত নয়। আগে চেক-আউটে বাকি রাতের সিট খালি হবে; no-show-তে ম্যানেজার ঠিক করবে।
হাজির তথ্য		
২৩	এজেন্সি নাম ছাড়া রুম আটকে রাখতে পারলে অপব্যবহারের ঝুঁকি। তথ্যের শেষ সময় বা অগ্রিম টাকা লাগবে?	কোটা দিয়ে সীমা, চেক-ইনের ৭ দিন আগে রিমাইন্ডার, স্বয়ংক্রিয় বাতিল নয়। অগ্রিম টাকার সিদ্ধান্ত ক্লায়েন্টের।
২৪	পাসপোর্টের ছবি বাইরের AI সার্ভিসে পাঠানো যাবে কি?	শুধু fallback হিসেবে, চুক্তি আর ডেটার রিজিয়ন যাচাই করে। আগে নিজের সার্ভারে MRZ আর OCR।
২৫	পাসপোর্টের ছবি কতদিন রাখা হবে? এনক্রিপশন কী কে রাখবে?	চেক-আউটের ১২ মাস পরে ছবি মোছা। কী ক্লাউডের key management সার্ভিসে।
২৬	পাসপোর্টের ন্যূনতম মেয়াদ কত হতে হবে?	চেক-ইন থেকে ৬ মাস (অ্যাডমিন সেটিং)। সৌদির বর্তমান নিয়ম ক্লায়েন্ট নিশ্চিত করবে।
অন্যান্য		
২৭	সার্ভার কোথায় রাখা হবে: সৌদির ভেতরে, নাকি বাহরাইন বা UAE-তে?	PDPL নিয়ে আইনি পরামর্শ না আসা পর্যন্ত সৌদির ভেতরের ক্লাউড রিজিয়ন।
২৮	অকুপেন্সি ক্যালেন্ডার কি রিসেপশনও দেখতে পারবে?	শুধু দিনের ভিউ, শুধু দেখা। পাসপোর্ট নম্বর নয়।
২৯	ট্রান্সপোর্ট সিস্টেম কোন ফেজে আসবে?	Phase 2-এর পরে, ops-console-এর অংশ হিসেবে।